Modelling Software-based Systems:
Correct by Construction Paradigm
Chapter *The Modelling Language Event-B*
Version November 9, 2026 at 2:03pm

Dominique MÉRY
2:03pm

November 9, 2026

Contents

Chapter 1. The Event-B Modelling Language 1
Dominique MÉRY

1.1. Introduction 1
1.2. Modelling Reactive Systems 2
1.3. Contexts and Machines in Event-B 15
1.3.1. Modelling sets, constants, axioms and theorems in a context D . 15
1.3.2. Modelling states and events in an abstract machine AM 20
1.3.2.1. The structure of abstract machine 20
1.3.2.2. Proof obligation th/THM 23
1.3.2.3. Proof obligation INITIALISATION/inv/INV 24
1.3.2.4. Proof obligations e/inv/INV and e/act/FIS 26
1.3.2.5. Proof obligations e/act/WD and INITIALISATION/act/WD 27
1.3.2.6. Status of an event 28
1.4. Refinement of Event-B machines 30
1.4.1. Elements on the refinement 32
1.4.2. Refinement machines in Event-B 38
1.4.2.1. The structure of refinement machine 38
1.4.3. Proof obligations for refinement 41
1.4.3.1. Proof obligations for INITIALISATION 43
1.4.3.2. Proof obligations for refinement: GRD, INV, SIM and WFIS 45
1.4.3.3. Additional proof obligations 48
1.4.3.4. Fusion of events 49
1.4.3.5. Semantics of Event-B 53
1.5. Overview of the Event-B Modelling Language 55
1.5.1. Playing with Event-B 55
1.5.1.1. Rule 1 of the abacus 57
1.5.1.2. Rule 2 of the abacus 59

1.6. Conclusion 66

1

The Event-B Modelling Language

Dominique MÉRY
[1]*Université de Lorraine, LORIA, France*

1.1. Introduction

The Event-B (**??**) method is based on a modelling language used to describe state-based models and safety properties of those state-based models. The originality of Event-B lies in its ability to enable incremental and proof-based modelling of *reactive systems*. The Event-B language contains both set notations and a first-order predicate calculus; it offers the possibility of defining models of reactive systems with *machines* and *contexts* and it includes the *refinement* relationship that allows us to follow an incremental development methodology. An Event-B machine is used to describe reactive systems, i.e. systems that react to their environment and its stimuli. An important property of such machines is that they maintain an inductive invariant that contains the set of reachable states of the current system. The Event-B language has been designed from the classical B (**?**) language and proposes a general framework for developing reactive systems, using a progressive approach to model by refinement. Refinement (**????**) is a relation linking two machines (or models), expressing that the less abstract model enriches the abstract one; the refinement of an abstract model by a concrete model means that the concrete model simulates the abstract model and that all invariance properties (inductive or not) of the abstract model are preserved in the concrete model. Event-B aims to express system models by an inductive invariant and by invariance properties, also called safety properties. In the course of the presentation of this language, we will see that liveness properties of the type $\leadsto$ (*leads to*) can

The Event-B Modelling Language,
coordinated by Dominique MÉRY. © ISTE Editions 2019.

also be implicitly integrated via the verification conditions for each $\leadsto$ property to be verified. We will also make a comparison with other state-based languages such as UNITY (**?**) or TLA$^+$ (**??**). We will look at the induction principles used in the Event-B method and then describe the elements that make up the syntax and the verification conditions called proof obligations.

1.2. Modelling Reactive Systems

Here are some basic elements for modelling what we will call a reactive system or simply a system. Modelling a system is done using well-known recipes, which we will recall first.

Definition 1 *(transition system)*

A transition system $\mathcal{ST}$ is given by a set of states Σ, a set of initial states $Init$ and a binary relation $\mathcal{R}$ on Σ.

A transition system may also be equipped with a set of terminal states $Term \subseteq \Sigma$, which identifies the states where the system terminates; this set can be empty, in which case the transition system does not terminate; this aspect can be used to model operating system programs that do not and should not terminate. We use the term *system* rather than *program* because we can describe more general entities than program, in the computer sense, but also because this formalism can be used for interactive, concurrent, distributed or hybrid applications[1]. Our definition is general, but we will apply it first to discrete systems. The idea is to observe transformations on the states of the system. Before modelling a system by a transition system, we must observe what constitutes the state of the system and induce transformations that operate on that state. A transformation is caused by an event: a sensor updating a temperature, a computer updating a variable, an actuator sending a signal to a controlled entity, or a biological evolution. An observation of a system S is based on the following points:

– a state $s \in \Sigma$ allows you to observe elements and reports on these elements, such as the number of people in the meeting room or the capacity of the room: $s(np)$ and $s(cap)$ are two natural numbers.

– a relationship between two states s and s' observes a transformation of the state s into a state s' and we will note $s \xrightarrow{R} s'$ which expresses the observation of a relationship R: $R = s(np) \in 0..s(cap) - 1 \wedge s'(np) = s(np) + 1 \wedge s'(cap) = s(cap)$ is an expression of R observing that one more person has entered the room.

1. We will see that so-called hybrid systems or applications require special treatment in terms of mathematical properties involving discrete and continuous domains, in particular real numbers and differential equations. These elements will be the subject of other chapters in Volume II of this book, which will complete the presentation of event-based modelling.

– a trace $s_0 \xrightarrow{R_0} s_1 \xrightarrow{R_1} s_2 \xrightarrow{R_2} s_3 \xrightarrow{R_3} \ldots \xrightarrow{R_{i-1}} s_i \xrightarrow{R_i} \ldots$ is a trace generated by the different observations $R_0, \ldots, R_i, \ldots$ which is infinite with possible repetitions.

We insist that we observe changes of state that correspond either to physical or biological phenomena or to artefactual structures such as a program, a service or a platform. An observation generally leads to the identification of a few possible transformations of the observed state, and the closed-model hypothesis follows naturally. One consequence is that there are visible transformations and invisible transformations. These invisible transformations of the state are expressed by an identity relation called event skip (or stuttering (**?**) or time-stretching (**?**)). Event-B modelling produces a closed model with a skip event modelling what is not visible in the observed state.

To express properties, a language of assertions $\mathcal{L}$ (or a language of formulae) is important. To simplify, we can take the assertion language $\mathcal{P}(\Sigma)$ (the power set of Σ) and $\varphi(s)$ (or $s \in \hat{\varphi}$) means that φ is true in s. The assertion language can be used to express state properties, but the assertion language in question may not be sufficiently expressive. In the context of program correctness, we will assume that assertion languages are sufficiently expressive (in Cook's sense), which means that the (state) properties required for completeness can be expressed in the language in question. Properties of a system S which interest us are the state properties expressing that *nothing bad can happen*. In other words, we wish to express state properties such as *the number of people in the meeting room is always smaller than the maximum allowed by law* or *the computer variable storing the number of wheel revolutions is sufficient and no overflow will happen*. A. van Gasteren and G. Tel (**?**) make a very important comment in the definition of what is *always true* and what is *invariant* and we choose to refer to state properties that are always true as *safety properties*. Safety properties are, for example, the partial correctness (PC) of an algorithm A with respect to its pre/post specifications, the absence of errors at runtime (RTE) ... Properties are expressed in the language $\mathcal{L}$ whose elements are combined by logical connectives and refer to the values of computer variables, called flexible variables (**?**). We assume that a system S is modelled by a set of states Σ, and that $\Sigma \stackrel{def}{=} \texttt{Var} \longrightarrow \text{D}$ where `Var` is the set of variables (or list of variables) of the system S and D is the domain of possible values of variables. The assertion language $\mathcal{L}$ can be used to define first-order predicate calculus formulas using set-theoretic operations ($\in$, $\subseteq$, $\cup$, ...) and operators ($\wedge$, $\vee$, ...). The interpretation of a formula P in a state $s \in \Sigma$ is denoted $[\![P]\!](s)$ or sometimes $s \in \hat{P}$. This hypothesis makes it possible to transfer from an assertion to the set of states validating this assertion. The definition of the validity of an assertion of $\mathcal{L}$ can be given in an inductive form of $[\![P]\!](s)$. The aim is not to give a complete definition but to give an idea of the interpretation of formulae, in order to expose elements specific to the Event-B language. A distinction is made between flexible variable symbols x and logical variable symbols v, and constant symbols c are used.

Example 1 *(interpretation of formulae)*

1) $[\![\mathtt{x}]\!](s) = s(\mathtt{x}) = x$*: x is the value of the variable* $\mathtt{x}$ *in* s*.*

2) $[\![\mathtt{x}]\!](s') = s'(\mathtt{x}) = x'$*: x' is the value of the variable* $\mathtt{x}$ *in* s'*.*

3) $[\![\mathtt{c}]\!](s) = c$ *is the value of* $\mathtt{c}$ *in* s*, in other words the value of the constant* $\mathtt{c}$ *in* s*.*

4) $[\![\varphi(x) \wedge \psi(x)]\!](s) = [\![\varphi(x)]\!](s)$ ***and*** $[\![\psi(x)]\!](s)$ *where* ***and*** *is the classical interpretation of symbol* $\wedge$ *according to the truth table.*

5) $[\![\mathtt{x} = 6 \wedge y = \mathtt{x} + 8]\!](s) \stackrel{def}{=} [\![\mathtt{x}]\!](s) = [\![6]\!](s)$ ***and*** $[\![y]\!](s) = [\![x]\!](s) + [\![8]\!](s) = (x = 6$ ***and*** $y = x + 8)$ *where y is a logical variable distinct from* $\mathtt{x}$ *and where* $[\![\mathtt{x}]\!](s) = s(\mathtt{x}) = x$*.*

We use notations which simplify the reference to states; thus, $[\![\mathtt{x}]\!](s)$ is the value of $\mathtt{x}$ in s and its value will be distinguished by the font used: $\mathtt{x}$ is the $\mathtt{tt}$ font of LaTeX and x is the math font of LaTeX. In this way, we can use the name of the variable $\mathtt{x}$ as its current value, i.e. x and $[\![\mathtt{x}]\!](s')$ is the value of $\mathtt{x}$ in s' and will be noted x'. So $[\![\mathtt{x} = 6]\!](s) \wedge [\![\mathtt{y} = \mathtt{x} + 8]\!](s')$ will be simplified to $x = 6 \wedge y' = x' + 8$. The consequence is that we can write the transition relation as a relation linking the state of the variables in s and the state of the variables in s' using the prime notation as defined by L. Lamport for TLA (**?**). We distinguish several types of variable depending on whether we are talking about the computer variable, its value or whether we are defining constants such as cap, the capacity of the room, or π, which designates the constant π. In the Event-B approach, a current observation refers to a current state for both endurant and perdurant information data in the sense of Dines Bjørner's approach (**?**).

Definition 2 *(flexible variable)*

A flexible variable $\mathtt{x}$ *is a name related to a perdurant information according to a state of the (current observed) system:*

– x *is the current value of* $\mathtt{x}$ *in other words the value at the observation time of* $\mathtt{x}$*.*

– x' *is the next value of* $\mathtt{x}$ *in other words the value at the next observation time of* $\mathtt{x}$*.*

– x_0 *is the initial value of* $\mathtt{x}$ *in other words the value at the initial observation time of* $\mathtt{x}$*.*

A logical variable x is a name related to an endurant entity designated by this name.

For a given system S, we will denote $\mathcal{V}(\mathrm{S})$ (resp. $\mathcal{V}ar(\mathrm{S})$) the set of logical (flexible) variables of the system S. The flexible variables are names used in writing the models and this set is used to distinguish these variables from other variables. When observing a system S, we wish to express relations between the flexible variables of

this system and we will note such an expression in the form of an assertion of the form $P(x)$ where x is the current value of the flexible variable $\mathtt{x}$ (or a list of flexible variables). The set $\mathcal{V}ar(S)$ is put into the form $\mathtt{x}$ i.e. $\mathtt{x} \in 1..n \rightarrowtail\!\!\!\!\!\rightarrow \mathcal{V}ar(\mathrm{S})$ where n is the number of flexible variables in the system S. We can write $\mathtt{x} = \mathtt{x}_1 \ldots \mathtt{x}_n$. We have defined the flexible variables which allow us to link the values of the D domain of the system we are observing. Observing a system S means determining the observed values of the D domain. Moreover, if a flexible variable $\mathtt{x}$ is used for modelling a system S, we can use notations as x, x' or x_0. We assume that we can simplify our process by using the expression x to designate the flexible variable $\mathtt{x}$, since we have defined the two sets of variables namely logical and flexible.

Definition 3 *(state property of a system)*

Let S *be a system whose flexible variables x are the elements of $\mathcal{V}ar(\mathrm{S})$. A property $P(x)$ of* S *is a logical expression involving freely the flexible variables x and whose interpretation is the set of values of x that satisfy it: $P(x)$ is true for x, if the value x satisfies $P(x)$.*

For each property $P(x)$, we can associate a subset of D *denoted $\hat{P}$ such that $P(x)$ is true for x if, and only if, $x \in \hat{P}$.*

Examples of properties are listed as follows:

– $P_1(x) \stackrel{def}{=} x \in 18..22$: x is a value between 18 and 22 and $\hat{P_1} = \{18, 19, 20, 21, 22\}$.

– $P_2(p) \stackrel{def}{=} p \subseteq PEOPLE \wedge card(p) \leq n$: p is a set of persons and that set has at most n elements and $\hat{P_2} = \{p \mid p \subseteq PEOPLE \wedge card(p) \leq n\}$. In this example, we use a constant n and a constant $PEOPLE$.

In our last example, we used $PEOPLE$ which represents a set of people and which we will therefore use to write our expressions. We denote by $s_1, s_2, \ldots, s_p$ the list of symbols corresponding to the symbols of the sets that make up the domain D.

Definition 4 *(basic set of a system* S*)*

The list of symbols $s_1, s_2, \ldots, s_p$ corresponds to the list of basic set symbols in the D *domain of S and $s_1 \cup \ldots \cup s_p \subseteq$* D.

Finally, to model a system S, you need symbols for constants, which correspond to endurant information data.

Definition 5 *(constants of system* S*)*

The list of symbols $c_1, c_2, \ldots, c_q$ corresponds to the list of symbols for the constants of S.

In fact, we are not using the classical partition of logic languages, which separates constant symbols on one side from function symbols on the other. We will give a few examples to get the idea across.

Example 2 *(examples of constant and set)*

– $fred$ is a constant and is linked to the set $PEOPLE$ using the expression $fred \in PEOPLE$ which means that $fred$ is a person from the set $PEOPLE$.

– aut is a constant which is used to express the table of authorisations associated with the use of vehicles. The expression $aut \subseteq PEOPLE \times CARS$ where $CARS$ denotes a set of cars means that, if a pair $p \mapsto c \in aut$, then p is authorized to use the car c.

The constants of S cover the basic constants and the functions of S. Each constant c of S must be defined by a list of expressions called axioms. The list of basic sets (resp. constants) is denoted s (resp. c).

Definition 6 *(axiom of system* S*)*

An axiom ax(s,c) of S *is a logical expression describing a constant or constants of* S *and can be defined as an expression depending on symbols of constants expressing a set-theoretical expression using symbols of sets and symbols of constants already defined.*

We give a few examples of axioms that can be defined:

Example 3 *(examples of axiom)*

– $ax1(fred \in PEOPLE)$: fred is a person from the set PEOPLE

– $ax2(suc \in \mathbb{N} \rightarrow \mathbb{N} \wedge (\forall i.i \in \mathbb{N} \Rightarrow suc(i) = i + 1))$: The function suc is the total function which associates any natural i with its successor.

*– $ax3(\forall A.A \subseteq \mathbb{N} \wedge 0 \in A \wedge suc[A] \subseteq A \Rightarrow \mathbb{N} \subseteq A)$: This axiom states the induction property for natural numbers. It follows from the characterisation of $\mathbb{N}$ as a least fixed point (Knaster-Tarski theorem) (**?**).*

– $ax4(\forall x.x = 2 \Rightarrow x + 2 = 1)$: This axiom poses an obvious problem of consistency and care should be taken not to use this kind of statement as axiom.

We have numbered axioms and we will use this numbering to define axioms of the system S. One assumption is that axioms are consistent but it should be checked by the user. For any system S, we will use a list of axioms to describe constants of S.

Definition 7 *(axiomatics for* S*)*

The list of axioms of S *is called the axiomatics of* S *and is denoted* $AX(s, c)$ *where* s *denotes the basic sets and* c *denotes the constants of* S.

Advice 1 *(consistency of the axiomatisation of* S*)*

Checking the consistency of $AX(s, c)$ *is an important part of modelling a system. It is quite easy to introduce inconsistency and tools such as Rodin provide the ProB tool based on the discovery of a model in the logical sense. However, this technique has its limits and you need to be very careful.*

We have defined an axiomatic system $AX(s, c)$ for the system S and we will now derive some properties from this system. These properties will be proved from this axiomatic system and will be the theorems for S.

Definition 8 *(theorem for* S*)*

A property $P(s, c)$ *is a theorem for* S*, if* $AX(s, c) \vdash P(s, c)$ *is a valid sequent.*

Theorems for S *are denoted by* $TH(\mathrm{S}, s, c)$.

We have shown how definitions of basic sets, constants, axioms and theorems are organised. The flexible variables have an essential quality, since they allow us to account for the state of the system under observation.

The letter s, used so far for states, denotes from now on the list of basic sets. Let σ, σ' be two states of S ($\sigma, \sigma' \in \mathcal{V}ar(\mathrm{S}) \longrightarrow \mathrm{D}$). $\sigma \xrightarrow{R} \sigma'$ will be written as a relation $\mathrm{R}(x, x')$ where x and x' designate values of x before and after the observation of R. We define what is an observed event on the *flexible variables* of the observed system S.

Definition 9 *(event)*

Let $\mathcal{V}ar(\mathrm{S})$ *be the set of flexible variables of* S*. Let* s *be the basic sets and* c *the constants of* S*. An event* e *for* S *is a relational expression of the form* $R(s, c, x, x')$ *denoted* $BA(e)(s, c, x, x')$.

This definition is borrowed directly from TLA (**?**) and simplifies the presentation of our preliminaries. On the other hand, we will not borrow the set-theoretic language of TLA$^+$ (**?**). We give examples of events:

Example 4 *(examples of event)*

– $BA(e_1)(x, x') \stackrel{def}{=} x' = x + 1$*:* e_1 *observes the increase of* $\boldsymbol{x}$ *by one unit.*

– $BA(e_2)(x, y, x', y') \stackrel{def}{=} x' + y' = x + y$*:* e_2 *observes that the values of* $\boldsymbol{x}$ *and* $\boldsymbol{y}$ *evolve so that the sum of the two variables is constant.*

Convention 1 *(meta-language of proofs)*

We choose to use logical expressions of the form $P \Rightarrow Q$ *or* $\forall x.P(x)$ *or* $\forall x \in E.P(x)$ *in the meta-language of proofs and formal expressions. This will allow us to present fundamental results on induction principles and verification conditions. We will then express the verification in a form closer to the logical tools used.*

Following an observation of a system S, a set of events E is identified and an event-based model of the system S is obtained.

Definition 10 *(event-based model of a system)*

Let $\mathcal{V}ar(\mathrm{S})$ *be the set of flexible variables of* S *denoted* x*. Let* s *be the list of basic sets of the system* S*. Let* c *be the list of constants of the system* S*. Let* D *be a domain containing sets* s*. An event-based model for a system* S *is defined by*

$$(AX(s, c), \{x\}, \mathrm{D}, Init(x), \{e_0, \ldots, e_n\})$$

where

– $AX(s, c)$ *is an axiomatic theory defining the sets, constants and static properties of these elements.*

– $Init(x)$ *defines the possible initial values of x.*

– $\{e_0, \ldots, e_n\}$ *is a finite set of events of* S *and* e_0 *is a particular event present in each event-based model defined by* $BA(e_0)(x, x') = (x' = x)$*.*

The event-based model is denoted $EM(s, c, \{x\}, \mathrm{D}, Init(x), \{e_0, \ldots, e_n\}) = (AX(s, c), \{x\}, \mathrm{D}, Init(x), \{e_0, \ldots, e_n\})$*.*

From this structure, we can define a relationship $Next(s, c, x, x')$ as follows $Next(s, c, x, x') \stackrel{def}{=} BA(e_0)(s, c, x, x') \vee \ldots \vee BA(e_n)(s, c, x, x')$. Modelling a

system involves giving the variable *x*, the predicate $Init(x)$ characterising the initial values of the variables and a relationship $Next(s, c, x, x')$ modelling the relationship between the values before and the values after. Safety properties express that *nothing bad can happen (***?***)*. For example, the value of the variable *x* is always between 0 and 567; the sum of the current values of the variables *x* and *y* is equal to the current value of the variable z. To continue our descriptions, we need to introduce the reflexive transitive closure of the relation $Next$, that is, the least relation $Next^\star$ satisfying:

$$Next^\star(s, c, x_0, x) \stackrel{def}{=} \begin{cases} \vee\ x = x_0 \\ \vee\ \exists\ xi \in \mathrm{D}.Next^\star(s, c, x_0, xi) \wedge Next(s, c, xi, x) \end{cases} \quad [1.1]$$

Definition 11 *(safety property)*

A property $P(x)$ is a safety property for the system S, if

$$\forall x_0, x \in \mathrm{D}.Init(x_0) \wedge Next^\star(s, c, x_0, x) \Rightarrow P(x).$$

The safety property uses a universal expression to quantify the possible values of the variable *x*. To demonstrate such a property, we can either check the property for every possible value of *x* in the domain D, provided that this set is finite, or use an abstraction of the domain Dor use an induction principle. For the verification for each possible value, we use an algorithm to calculate the values reachable from an initial state. This technique of calculating reachable values is often used and is the basis of automatic verification techniques such as *model-checking* (**???**). We consider an inductive technique to prove safety properties. We observe the logical equivalence between the two formulas:

$$\forall x_0, x \in \mathrm{D}.Init(x_0) \wedge Next^\star(s, c, x_0, x) \Rightarrow P(x) \quad [1.2]$$

$$\forall x \in \mathrm{D}.(\exists x_0 \in \mathrm{D}.Init(x_0) \wedge Next^\star(s, c, x_0, x)) \Rightarrow P(x) \quad [1.3]$$

Thus, the second equation expresses that the accessible values are safe values with respect to $P(x)$ and gives us the key to the induction principle to be implemented. In fact, the property $(\exists x_0 \in \mathrm{D}.Init(x_0) \wedge Next^\star(s, c, x_0, x))$ expresses that x is a reachable value with respect to the $Next(s, c, x_0, x)$ relationship and defines a least fixed point which is an inductive property.

Propery 1 *(induction principle)*

A property $P(x)$ is a safety property for S if, and only if, there exists a property $I(x)$ such that

$$\forall x, x' \in \mathrm{D}. \begin{cases} (1)\ \ Init(x) \Rightarrow I(x) \\ (2)\ \ I(x) \Rightarrow P(x) \\ (3)\ \ I(x) \wedge Next(s, c, x, x') \Rightarrow I(x') \end{cases}$$

The property $I(x)$ is called an (inductive) invariant and is a particular property that is stronger than the safety property $P(x)$. The justification for this induction principle is quite simple. This property justifies the method of proof by induction better known as the Floyd/Hoare method (**??**), devised by Turing in 1949 (**?**). This property gives a form of induction that must be reduced to more familiar forms. P. and R. Cousot (**????**) give a complete summary of the principles of induction equivalent to this principle of induction. We apply these results to the case of event-based models and obtain an expression for the definition of a safety property. If we transform the properties, we obtain a form closer to what we will use in the following and closer to the notions of event-based models.

Propery 2 *(equivalence of induction principles)*

The following two statements are equivalent:

(I) There exists a state property $I(x)$ such that:

$$\forall x, x' \in \mathrm{D}. \begin{cases} (1)\ \ Init(x) \Rightarrow I(x) \\ (2)\ \ I(x) \Rightarrow P(x) \\ (3)\ \ I(x) \wedge Next(s, c, x, x') \Rightarrow I(x') \end{cases}$$

(II) There exists a state property $I(x)$ such that:

$$\forall x, x' \in \mathrm{D}. \begin{cases} (1)\ \ Init(x) \Rightarrow I(x) \\ (2)\ \ I(x) \Rightarrow P(x) \\ (3)\ \ \forall i \in \{0, \ldots, n\} : I(x) \wedge BA(e_i)(s, c, x, x') \Rightarrow I(x') \end{cases}$$

We have thus given an explanation of the induction rule which is used in the Floyd–Hoare method (**???**); this rule makes it possible to distinguish the invariance properties, which are proved directly by an induction step, from the more general safety properties, which require a stronger inductive invariant in order to be proven. The Event-B method implements these two types of property using the INVARIANTS clause for invariants and the THEOREMS clause for safety properties. We deduce that any invariance property is a safety property. We will describe the Event-B language and the method for incremental development of event-based models. The following verification conditions are derived from the above properties.

Definition 12 *(verification conditions for a system* S *and its invariance and safety properties)*

Let $EM(S) = (AX(s,c), \{x\}, \mathrm{D}, Init(x), \{e_0, \ldots, e_n\})$ *be an event-based model for system S.* $EM(S)$ *is a valid event-based model for* S*, if the following verification conditions for a system* S *and its invariance properties* $I(x)$ *and safety properties* $A(x)$ *are valid (the safety property* $P(x)$ *of the previous properties is denoted* $A(x)$ *from now on):*

– $AX(s,c) \vdash \forall x \in \mathrm{D}.Init(x) \Rightarrow I(x)$

– *For any event* **e** *in* S*,* $AX(s,c) \vdash \forall x, x' \in \mathrm{D}.I(x) \wedge BA(e)(x,x') \Rightarrow I(x')$

– $AX(s,c) \vdash \forall x \in \mathrm{D}.I(x) \Rightarrow A(x)$

We have used expressions of the form $AX(s,c) \vdash \forall x \in \mathrm{D}.P(x)$ and these expressions are equivalent to $AX(s,c) \vdash \forall x.x \in \mathrm{D} \Rightarrow P(x)$. Considering that x has no free occurrences in $AX(s,c)$, we can simplify the expression into the form $AX(s,c), x \in \mathrm{D} \vdash P(x)$ and meaning that x is a value of D. The expression $x \in \mathrm{D}$ constitutes a typing assumption for x and this information is made explicit directly or indirectly in the invariant $I(s,c,x)$ and in the initial conditions $Init(s,c,x)$. We assume that the information $x \in \mathrm{D}$ is expressed in the initial conditions and in the invariant. The conditions of definition 12 are simplified into the following new conditions:

– $AX(s,c) \vdash Init(x) \Rightarrow I(x)$

– For any event **e** in S, $AX(s,c) \vdash I(x) \wedge BA(e)(x,x') \Rightarrow I(x')$

– $AX(s,c) \vdash I(x) \Rightarrow A(x)$

Before continuing with this presentation dedicated to Event-B , we would like to remind you that Event-B is a language that supports an approach to developing and modelling systems that are correct by construction. Consequently, the event-based model of the system S $EM(S) = (AX(s,c), \{x\}, \mathrm{D}, Init(x), \{e_0, \ldots, e_n\})$ (definition 10) is characterised by the existence of fixed points on the complete lattice $(\mathbb{P}(\mathrm{D}), \subseteq)$.

Propery 3 *(fixed-point characterization of invariants and safety properties)*

Let D *be the value domain of the event-based model* $EM(S) = (AX(s,c), \{x\}, \mathrm{D}, Init(x), \{e_0, \ldots, e_n\})$.

Let $Next(s,c,x,x') \stackrel{def}{=} BA(e_0)(s,c,x,x') \vee \ldots \vee BA(e_n)(s,c,x,x')$. *Let* $INIT = \{d | d \in \mathrm{D} \wedge Init(d)\}$.

– $(\mathbb{P}(\mathrm{D}), \subseteq)$ *is a complete lattice.*

– *The function* $F = \lambda X \in \mathbb{P}(\mathrm{D}).(INIT \cup Next[X])$ *is monotonically increasing and the equation* $X = F(X)$ *has a non-empty set of solutions forming a complete lattice.*

– A set $I \in \mathbb{P}(\mathrm{D})$ satisfies $F(I) \subseteq I$ if, and only if, $INIT \subseteq I$ and $Next[I] \subseteq I$; in this case, $lfp(F) \subseteq I$. This holds in particular for every solution of $X = F(X)$.

– For any safety property $P(x)$ for system S*, $lfp(F) \subseteq \{d | d \in \mathrm{D} \wedge P(d)\}$*

A consequence of this property is that a *canonical* invariant can be associated with any event-based model and this invariant is operationally defined by the least fixed-point of the function F. This approach is generally used in the verification of a system. In the case of the Event-B language, we are interested in a correctness-by-construction approach. Consequently, the problem is to define a model equipped with an invariant that will be verified and this method is carried out incrementally with the help of refinement. The sets X such that $F(X) \subseteq X$ (the pre-fixed points of F), and not only the solutions of $X = F(X)$, correspond to inductive invariants and are useful for showing that a property $P(x)$ is a safety property. The triptych $(EM(S), I, P)$ puts forward an event-based model $EM(S)$ whose (inductive) invariant allows the safety property $P(x)$ to be verified (by discharging the verification conditions for the invariance of I and the safety of $P(x)$).

Before moving on to the presentation of structures for modelling reactive systems, we are interested in the notion of event and in the so-called feasibility conditions associated with model verification. J.-R. Abrial (**?**) bases the verification of abstract machines on the wp calculus and uses the notation $[S]P$ (generalised substitution), to express that *S establishes P*. This expression also means that S terminates in *P* and the termination predicate is denoted trm(e)(s,c,x). In our case, S is an event e and we recall the definition $BA(\mathsf{e})(s,c,x,x') \stackrel{def}{=} \exists u.G(u,s,c,x) \wedge BAP(u,s,c,x,x')$ where $G(u,s,c,x)$ is a guard and $BAP(u,s,c,x,x')$ is a before-after relation$BA(\mathsf{e})(s,c,x,x')$.

By definition, $\mathsf{trm}(e)(s,c,x)$ is the predicate $[e]\,true$. The event is $e \mathrel{\widehat{=}}$ **any** u **where** $G(u,s,c,x)$ **then** $x :| BAP(u,s,c,x,x')$ **end**. Applying the rule for the unbounded choice and the guarded substitution, we obtain

$$[e]\,true \;\equiv\; \forall u \cdot \big(G(u,s,c,x) \Rightarrow [x :| BAP(u,s,c,x,x')]\,true\big).$$

The rule for the non-deterministic assignment, $[x :| P(x,x')]\,R \equiv \forall x' \cdot (P(x,x') \Rightarrow R[x'/x])$, then gives

$$\mathsf{trm}(e)(s,c,x) \;\equiv\; \forall u \cdot \big(G(u,s,c,x) \Rightarrow \forall x' \cdot (BAP(u,s,c,x,x') \Rightarrow true)\big).$$

Since $P \Rightarrow true$ is valid for every P, the inner formula reduces to $true$, and so does the whole expression: $\mathsf{trm}(e)(s,c,x) \equiv true$. In other words, an Event-B event always terminates, assuming its well-definedness conditions hold. However, its feasibility,

denoted $\mathsf{FIS}(e)$, can fail. An event e is *feasible* if, whenever its guard holds, its action can actually produce at least one after-state. This is the proof obligation

$$\mathsf{FIS}: \quad AX(s,c),\ I(s,c,x),\ G(u,s,c,x) \ \vdash\ \exists x' \cdot BAP(u,s,c,x,x').$$

The condition must hold for *every* value of the parameter u allowed by the guard, but only in states that satisfy the invariant. In closed form:

$$I(s,c,x) \Rightarrow \mathsf{FIS}(e)(s,c,x)$$
$$\mathsf{FIS}(e)(s,c,x) \mathrel{\widehat{=}} \forall u \cdot \big(G(u,s,c,x) \Rightarrow \exists x' \cdot BAP(u,s,c,x,x')\big).$$

It rules out the *hidden* miracle: a guard that holds while the action produces no after-state, so that the event looks enabled when it is not. Under this condition, the written guard $\exists u \cdot G(u,s,c,x)$ coincides with the actual guard

$$\mathsf{fis}(e)(s,c,x) \ \equiv\ \exists u \cdot \big(G(u,s,c,x) \wedge \exists x' \cdot BAP(u,s,c,x,x')\big).$$

This is what makes the deadlock-freedom proofs, the refinement proofs and the execution of the model sound. Feasibility is independent of invariant preservation (INV). Together, the two obligations form the weakest-precondition condition

$$I(s,c,x) \Rightarrow \mathsf{FIS}(e)(s,c,x) \wedge [e]\, I(s,c,x).$$

This discussion leads us to distinguish two notions of feasibility: the *global* feasibility $\mathsf{fis}(e)$, which is the actual guard of e, and the *per-parameter* feasibility $\mathsf{FIS}(e)$, required by the proof obligation FIS, which rules out hidden miracles.

Propery 4 *(wp and relational styles) Let $P(s,c,x)$ be a state property and let* $\mathsf{e} \mathrel{\widehat{=}}$ ***any*** *u* ***where*** *$G(u,s,c,x)$* ***then*** *$x : |BAP(u,s,c,x,x')$* ***end****, that is,* $\mathsf{e} = @u \cdot \big(G(u,s,c,x) \Longrightarrow x : |BAP(u,s,c,x,x')\big)$.

1) $[\mathsf{e}]\, P(s,c,x) \ \equiv\ \forall u \cdot \big(G(u,s,c,x) \Rightarrow [x : |BAP(u,s,c,x,x')]\, P(s,c,x)\big)$
$\equiv\ \forall u \cdot \big(G(u,s,c,x) \Rightarrow \forall x' \cdot (BAP(u,s,c,x,x') \Rightarrow P(s,c,x'))\big)$*;*

2) we define $\mathsf{FIS}(\mathsf{e})(s,c,x) \mathrel{\widehat{=}} \forall u \cdot \big(G(u,s,c,x) \Rightarrow \exists x' \cdot BAP(u,s,c,x,x')\big)$*;*

3) under the axioms $AX(s,c)$, the following two properties are equivalent (free variables are universally quantified):

a) $\vdash\ I(s,c,x) \Rightarrow \mathsf{FIS}(\mathsf{e})(s,c,x) \wedge [\mathsf{e}]\, I(s,c,x)$ *(wp)*

b) $\begin{cases} \vdash\ I(s,c,x) \wedge G(u,s,c,x) \Rightarrow \exists x' \cdot BAP(u,s,c,x,x') \\ \vdash\ I(s,c,x) \wedge G(u,s,c,x) \wedge BAP(u,s,c,x,x') \Rightarrow I(s,c,x') \end{cases}$ *(relational)*

Item 1 is a direct application of the rules of the generalized substitution calculus of J.-R. Abrial (**?**), and item 2 is a definition. Item 3 states that two formulations of the correctness of an event e with respect to an invariant I are equivalent. The predicate transformer [e] defines the weakest precondition of e for a given postcondition. For an event, termination always holds, but [e] says nothing about feasibility: a parameter value for which the action yields no after-state satisfies every postcondition vacuously. Feasibility must therefore appear explicitly, as the conjunct $\mathsf{FIS}(\mathsf{e})$ in the *wp* form and as the first sequent in the relational form. J.-R. Abrial (**?**) gave a complete study of [e], and he introduced the foundational ideas of Event-B in his seminal talk at B96 (**?**). The Atelier B platform (**?**) generates verification conditions in the *wp* style, whereas the Rodin platform (**?**) uses the relational style.

Explanation 1 *We write I, I', G and BAP for $I(s,c,x)$, $I(s,c,x')$, $G(u,s,c,x)$ and $BAP(u,s,c,x,x')$, and we read each line as its universal closure.*

(relational)

$\Leftrightarrow$ { x' *is not free in* $I \wedge G$ }

$I \wedge G \Rightarrow \exists x' \cdot BAP \quad$ *and* $\quad I \wedge G \Rightarrow \forall x' \cdot (BAP \Rightarrow I')$

$\Leftrightarrow$ { $[x : |BAP]\, R \Leftrightarrow \forall x' \cdot (BAP \Rightarrow R[x'/x])$ }

$I \wedge G \Rightarrow \exists x' \cdot BAP \quad$ *and* $\quad I \wedge G \Rightarrow [x : |BAP]\, I$

$\Leftrightarrow$ { $(A \wedge B \Rightarrow C) \Leftrightarrow (A \Rightarrow (B \Rightarrow C))$; $(A \Rightarrow B) \wedge (A \Rightarrow C) \Leftrightarrow (A \Rightarrow B \wedge C)$ }

$I \Rightarrow \big(G \Rightarrow \exists x' \cdot BAP\big) \wedge \big(G \Rightarrow [x : |BAP]\, I\big)$

$\Leftrightarrow$ { u *is not free in* I; $\forall u \cdot (A \wedge B) \Leftrightarrow \forall u \cdot A \wedge \forall u \cdot B$ }

$I \Rightarrow \forall u \cdot \big(G \Rightarrow \exists x' \cdot BAP\big) \wedge \forall u \cdot \big(G \Rightarrow [x : |BAP]\, I\big)$

$\Leftrightarrow$ { *definition of* FIS; $[@u \cdot S]R \Leftrightarrow \forall u \cdot [S]R$; $[G \Longrightarrow S]R \Leftrightarrow (G \Rightarrow [S]R)$; *definition of* e }

$I \Rightarrow \mathsf{FIS}(\mathsf{e}) \wedge [\mathsf{e}]\, I \;=$ *(wp)*

A consequence of this result is to allow a definition of invariant preservation according to two modes of implementation (Atelier B (**?**) and Rodin (**?**)). We will break down the definition of preservation in the form that separates verification into an induction step and a proof of feasibility. In particular, it defines verification conditions (PO(e)) using these elements as follows.

Definition 13 *(verification condition* init*)*

init *is an initialisation event and we assume that it is defined as follows:*

$\mathsf{init} \stackrel{def}{=}$ **begin** $x : |initBAP(s,c,x')$ **end**.

The verification condition for init *(initialisation) denoted* PO(init) *is defined by*
$\begin{cases} (FIS)\ AX(s,c) \vdash \exists x'.initBAP(s,c,x') \\ (INV)\ AX(s,c), initBAP(s,c,x') \vdash I(s,c,x') \end{cases}$

Definition 14 *(verification condition* e*)*

The verification condition for e *(event* e*) denoted* PO(e) *is defined by* $AX(s,c) \vdash I(s,c,x) \Rightarrow \mathsf{FIS}(e)(s,c,x) \wedge [e]\, I(s,c,x)$*, which is equivalent (Property 4) to the two sequents:*

$$\begin{cases} (FIS)\ AX(s,c), I(s,c,x), G(u,s,c,x) \vdash \exists x'.BAP(u,s,c,x,x') \\ (INV)\ AX(s,c), I(s,c,x), G(u,s,c,x), BAP(u,s,c,x,x') \vdash I(s,c,x') \end{cases}$$

This definition puts forward two conditions INV and FIS which must be verified and which ensure that $I(s,c,x)$ is preserved. We have specified the elements we are going to use to present the implementation in the Event-B language.

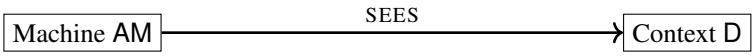
Convention 2 *(label for formal texts)*

In the definition of the Event-B language, the implementation is based on a labelling of each formal text (axiom, theorem, guard, action) and this labelling is noted $\ell(text)$*. This means that the expression* $\ell(text) : text$ *appears in the modelling text. This labelling makes it possible to refer to elements of the model.*

1.3. Contexts and Machines in Event-B

1.3.1. *Modelling sets, constants, axioms and theorems in a context* D

Event-B is organised according to context and machine structures. These structures form the basic structures for the definition of an experimental model. Schematically, the relationship between a machine AM and a context D is expressed by the link sees which is expressed in the machine AM.



An Event-B context brings together the definitions of the endurant entities of the system S to be developed and therefore modelled. Now we refer to the document (**?**) presenting the Event-B mathematical language constituting the core of the Event-B language.

CONTEXT D
EXTENDS AD
SETS
$S_1, \ldots, S_n$
CONSTANTS
$C_1, \ldots, C_m$
AXIOMS
$ax_1 : P_1(S_1, \ldots, S_n, C_1, \ldots, C_m)$
$\ldots$
$ax_p : P_p(S_1, \ldots, S_n, C_1, \ldots, C_m)$
THEOREMS
$th_1 : Q_1(S_1, \ldots, S_n, C_1, \ldots, C_m)$
$\ldots$
$th_q : Q_q(S_1, \ldots, S_n, C_1, \ldots, C_m)$

– Sets s $(S_1, \ldots, S_n)$ are declared in the SETS clause.

– Constants c $(C_1, \ldots, C_m)$ are declared in the CONSTANTS clause.

– Axioms are listed in the AXIOMS clause and define properties of constants.

– Theorems are properties declared in the THEOREMS clause and must be proved from axioms.

– The context defines a logical-mathematical theory which must be consistent.

– The EXTENDS clause extends the context and therefore extends the theory defined by the context of this clause.

– $AX(s, c)$ designates the list of axioms corresponding to the sets s and constants c.

– $TH(s, c)$ designates the list of theorems corresponding to the sets s and constants c.

The clause THEOREMS does not exist in the current version of the Rodin platform, but it does make it possible to separate the axioms from the theorems. The Rodin platform uses a theorem or axiom indication and produces an italicised version for statements that are theorems, but axioms and theorems are placed under the same AXIOMS clause. This representation may confuse beginners.

Each Event-B expression $expr$ must be well defined, and a verification condition, called *expr/WD*, is systematically produced from the text of the property to be proved; the verification condition establishing that *expr* is well defined is denoted by *expr/WD*. Intuitively, an expression *expr* is well defined, if it obeys certain rules of use, and it is denoted *WD(expr)*. We have given the tables that define the predicate *WD(expr)* according to the syntax of *expr* and according to the classes of expressions that are predicates, relations ...

From the tables 1.1, 1.2 and 1.3, we can simply establish the verification condition to be proved, denoted expr/WD.

Proof Obligation expr/WD

$AX(s, c) \vdash WD(expr)$

---

e	WD(e)
$P \wedge Q, P \Rightarrow Q$	$WD(P) \wedge (P \Rightarrow WD(Q))$
$P \vee Q$	$WD(P) \wedge (P \vee WD(Q))$
$P \Leftrightarrow Q$	$WD(P) \wedge WD(Q)$
$\neg P$	$WD(P)$
$\forall L.P$, $\exists L.P$	$\forall L.WD(P)$
$\top$, $\bot$	$\top$
$\text{finite}(E)$	$WD(E)$
$partition(E_1, E_2, \ldots, E_n)$	$WD(E_1) \wedge WD(E_2) \wedge \ldots \wedge WD(E_n)$
E op F with op $\in \{=, \neq, \in, \notin, \subset, \not\subset, \subseteq, \not\subseteq\}$	$WD(E) \wedge WD(F)$

Table 1.1. *Well-definedness (WD) for predicates*

e	WD(e)
$F(E)$	$\begin{pmatrix} WD(E) \wedge WD(F) \\ E \in dom(F) \wedge F \in S \pfun T \\ F \subseteq S \times T \end{pmatrix}$
$E[F], E \mapsto F, E \leftrightarrow F, E \nrightarrow F, E \nleftrightarrow F$ $E \pinj F, E \rightarrow F, E \pfun F, E \tinj F, E \tfun F$ $E \surj F, E \psurj F, E \tsurj F, E \cup F, E \cap F$ $E \setminus F, E \times F, E \otimes F, E \parallel F, E ; F, E \circ F$ $E \domres F, E \ndomres F, E \ranres F, E \nranres F, E \bcomp F$ $E..F, E + F, E - F, E * F$	$WD(E) \wedge WD(F)$
E/F	$WD(E) \wedge WD(F) \wedge F \neq 0$
$E \bmod F$	$WD(E) \wedge WD(F) \wedge 0 \leq E \wedge 0 < F$
E ^ F	$WD(E) \wedge 0 \leq E \wedge WD(F) \wedge 0 \leq F$
$-E, E^{-1}, \mathbb{P}(E), \mathbb{P}_1(E)$ $\text{dom}(E), \text{ran}(E), \text{union}(E)$	$WD(E)$
$\text{card}(E)$	$WD(E) \wedge \text{finite}(E)$
$\text{inter}(E)$	$WD(E) \wedge E \neq \varnothing$
$\min(E)$	$WD(E) \wedge E \neq \varnothing \wedge (\exists v. \forall u. u \in E \Rightarrow v \leq u)$
$\max(E)$	$WD(E) \wedge E \neq \varnothing \wedge (\exists v. \forall u. u \in E \Rightarrow v \geq u)$

Table 1.2. *Well-definedness (WD) for unary and binary expressions*

A few examples will illustrate the $WD(expr)$ notation and enable you to understand the verification conditions produced by the tool Rodin.

Exemple 1 (Examples for $WD(expr)$)

– $E1 = \forall k \cdot k \in \mathbb{N} \Rightarrow 2 * s(k) = k * k + k$, where s is the sum function of the context given below, and not the list of basic sets.

e	WD(e)
$\lambda P.Q \mid E$	$\forall \mathcal{F}_P . WD(Q) \wedge (Q \Rightarrow WD(E))$
$\bigcup L.P \mid E$ $\{L.P \mid E\}$	$\forall L . WD(P) \wedge (P \Rightarrow WD(E))$
$\bigcup E \mid P$ $\{E \mid P\}$	$\forall \mathcal{F}_E . WD(P) \wedge (P \Rightarrow WD(E))$
$\bigcap L.P \mid E$	$\forall L . WD(P) \wedge (P \Rightarrow WD(E))$ $\wedge$ $\exists L.P$
$\bigcap E \mid P$	$\forall \mathcal{F}_E . WD(P) \wedge (P \Rightarrow WD(E))$ $\wedge$ $\exists \mathcal{F}_E . P$
$\mathrm{bool}(P)$	$WD(P)$
$\{E_1, \ldots, E_n\}$	$WD(E_1) \wedge \ldots \wedge WD(E_n)$
$I, \mathbb{Z}, \mathbb{N}, \mathbb{N}_1, pred, succ, \mathrm{BOOL}$ $\mathrm{TRUE}, \mathrm{FALSE}, \varnothing, \mathrm{prj}_1, \mathrm{prj}_2, \mathrm{id}, n$	$\top$

Table 1.3. *WD for other expressions*

We simply apply the transformations of the tables 1.1, 1.2 and 1.3:
$WD(\forall k \cdot k \in \mathbb{N} \Rightarrow 2 * s(k) = k * k + k)$
$= \forall k . WD(k \in \mathbb{N} \Rightarrow 2 * s(k) = k * k + k)$
$= \forall k . WD(k \in \mathbb{N}) \wedge (k \in \mathbb{N} \Rightarrow WD(2 * s(k) = k * k + k))$
$= \forall k . WD(k \in \mathbb{N}) \wedge (k \in \mathbb{N} \Rightarrow WD(2 * s(k)) \wedge WD(k * k + k))$
$= \forall k . WD(k \in \mathbb{N}) \wedge (k \in \mathbb{N} \Rightarrow WD(2) \wedge WD(s(k)) \wedge WD(k * k) \wedge WD(k))$
$= \forall k . WD(k) \wedge WD(\mathbb{N}) \wedge (k \in \mathbb{N} \Rightarrow WD(2) \wedge WD(s(k)) \wedge WD(k * k) \wedge WD(k))$
$= \forall k . \top \wedge \top \wedge (k \in \mathbb{N} \Rightarrow \top \wedge WD(s(k)) \wedge WD(k * k) \wedge WD(k))$
$= \forall k . (k \in \mathbb{N} \Rightarrow WD(s(k)) \wedge WD(k * k) \wedge WD(k))$
$= \forall k . (k \in \mathbb{N} \Rightarrow WD(s(k)) \wedge WD(k) \wedge WD(k) \wedge WD(k))$
$= \forall k . (k \in \mathbb{N} \Rightarrow WD(s(k)) \wedge \top \wedge \top \wedge \top)$
$= \forall k . (k \in \mathbb{N} \Rightarrow WD(s(k)))$
$= \forall k . (k \in \mathbb{N} \Rightarrow WD(s) \wedge WD(k) \wedge k \in dom(s) \wedge s \in \mathbb{Z} \nrightarrow \mathbb{Z} \wedge s \subseteq \mathbb{Z} \times \mathbb{Z})$
$= \forall k . (k \in \mathbb{N} \Rightarrow \top \wedge \top \wedge k \in dom(s) \wedge s \in \mathbb{Z} \nrightarrow \mathbb{Z} \wedge s \subseteq \mathbb{Z} \times \mathbb{Z})$
$= \forall k . (k \in \mathbb{N} \Rightarrow k \in dom(s) \wedge s \in \mathbb{Z} \nrightarrow \mathbb{Z} \wedge s \subseteq \mathbb{Z} \times \mathbb{Z})$
We obtain the condition $WD(E1)$:
$WD(\forall k \cdot k \in \mathbb{N} \Rightarrow 2 * s(k) = k * k + k) = \forall k . (k \in \mathbb{N} \Rightarrow k \in dom(s) \wedge s \in \mathbb{Z} \nrightarrow \mathbb{Z} \wedge s \subseteq \mathbb{Z} \times \mathbb{Z})$

– $E2 \stackrel{def}{=} s(0) = 0$

$WD(s(0) = 0)$
$= WD(s(0)) \land WD(0)$
$= WD(s(0)) \land \top$
$= WD(s(0))$
$= WD(s) \land WD(0) \land 0 \in dom(s) \land s \in \mathbb{Z} \pfun \mathbb{Z} \land s \subseteq \mathbb{Z} \times \mathbb{Z}$
$= \top \land \top \land 0 \in dom(s) \land s \in \mathbb{Z} \pfun \mathbb{Z} \land s \subseteq \mathbb{Z} \times \mathbb{Z}$
$= 0 \in dom(s) \land s \in \mathbb{Z} \pfun \mathbb{Z} \land s \subseteq \mathbb{Z} \times \mathbb{Z}$
We derive the condition $WD(E2)$ using the same rules:
$WD(s(0) = 0) = 0 \in dom(s) \land s \in \mathbb{Z} \pfun \mathbb{Z} \land s \subseteq \mathbb{Z} \times \mathbb{Z}$

The verification condition th/THM is quite simple and sometimes requires interaction with the proof assistant. In this obligation, $Th(s, c)$ denotes the theorems of the context that precede *th*.

Proof Obligation th/THM

$AX(s, c), Th(s, c) \vdash th$

To conclude this presentation, we present an example of a context in the field of arithmetic. This allows us to pose a problem that will be used to illustrate the different notations and concepts.

Example 5 *(context for the sum of even or odd natural numbers)*

The problem is to calculate the sum $s(n)$ of the natural numbers between 0 and a given integer n, and this sum $s(n)$ is fairly easy to calculate using the formula classically used in elementary maths books: $\forall n.n \in \mathbb{N} \Rightarrow s(n) = \sum_{k=0}^{n} k = n * (n + 1)/2$ or $2 * s(n) = n * (n + 1)$. The problem is to propose an algorithm which calculates this sum and which respects the correctness property explicitly stated by the relation $2 * s(n) = n * (n + 1)$. We will also calculate the sum of the even integers $es(n)$ and the sum of the odd integers $os(n)$. The problem is to calculate the functions s, es, os, but we need to define these functions and establish a number of inductive properties.

The first step is to define the axiomatic properties (axm1–axm7) of the necessary sets and constants. The axiom axm7 is an axiomatic expression of induction for the domain of naturals and it will facilitate our proofs by induction that we will have to establish in the section THEOREMS.

AXIOMS

$axm1 : n \in \mathbb{N}$

$axm2 : s \in \mathbb{N} \to \mathbb{N} \land os \in \mathbb{N} \to \mathbb{N} \land es \in \mathbb{N} \to \mathbb{N}$

$axm3 : es(0) = 0 \land os(0) = 0 \land s(0) = 0$

$axm4 : \forall i, l \cdot i \in \mathbb{N} \land l \in \mathbb{N} \land i = 2 * l$
$\Rightarrow s(i+1) = s(i) + i + 1 \land es(i+1) = es(i) \land os(i+1) = os(i) + i + 1$

$axm5 : \forall i, l \cdot i \in \mathbb{N} \land l \in \mathbb{N} \land i = 2 * l + 1$
$\Rightarrow s(i+1) = s(i) + i + 1 \land es(i+1) = es(i) + i + 1 \land os(i+1) = os(i)$

$axm6 : suc \in \mathbb{N} \to \mathbb{N} \land (\forall i \cdot i \in \mathbb{N} \Rightarrow suc(i) = i + 1)$

$axm7 : \forall A \cdot A \subseteq \mathbb{N} \land 0 \in A \land suc[A] \subseteq A \Rightarrow \mathbb{N} \subseteq A$

THEOREMS

$th1 : \forall i \cdot i \in \mathbb{N} \Rightarrow s(i+1) = s(i) + i + 1$

$th2 : \forall u, v \cdot u \in \mathbb{N} \land v \in \mathbb{N} \land 2 * u = v \Rightarrow u = v/2$

$th3 : \forall k \cdot k \in \mathbb{N} \Rightarrow 2 * s(k) = k * k + k$

$th4 : \forall k \cdot k \in \mathbb{N} \Rightarrow s(k) = (k * k + k)/2$

$th5 : \forall k \cdot k \in \mathbb{N} \Rightarrow es(2 * k) = 2 * s(k)$

$th6 : \forall k \cdot k \in \mathbb{N} \Rightarrow es(2 * k + 1) = 2 * s(k)$

$th7 : \forall k \cdot k \in \mathbb{N} \land k \neq 0 \Rightarrow os(2 * k) = k * k$

$th8 : \forall k \cdot k \in \mathbb{N} \Rightarrow os(2 * k + 1) = (k + 1) * (k + 1)$

Theorems th1–th8 are based on fairly elementary properties; th3 and th4 are the properties stated in the problem. These theorems are proved using the Rodin tool with the help of the axiom axm7 expressing the induction rule over naturals. They are an important element in linking the inductive definition of sequences and the property expected for this sequence. Thus, the inductive definition of the sequence s and the property are linked as follows: $\forall i.i \in \mathbb{N} \Rightarrow s(i) = i * (i+1)/2$; the role of the inductive definition of s is to give a method of calculation. We will use this method in the illustration of refinement.

1.3.2. *Modelling states and events in an abstract machine* AM

1.3.2.1. *The structure of abstract machine*

In Section 1.2, we introduced the notion of an event-based model of a system by the definition 10.

MACHINE AM
REFINES M
SEES D
VARIABLES x
INVARIANTS
$inv_1 : I_1(s, c, x)$
...
$inv_r : I_r(s, c, x)$
THEOREMS
$th_1 : SAFE_1(s, c, x)$
...
$th_h : SAFE_h(s, c, x)$
VARIANT
$var : varexp(s, c, x)$
EVENTS
Event initialisation
begin
$x : |initBAP(s, c, x')$
end
...
Event e
any u where
$G(u, s, c, x)$
then
$x : |BAP(u, s, c, x, x')$
end
...
END

Event-B has the abstract machine structure to represent such a state-based model. We give the syntax of an abstract machine AM which uses a context D and which describes a state observed by the variable x. This state variable x is characterised by a set of invariants $I_j(s, c, x), j \in 1..r$ and by a set of safety properties $SAFE_i(s, c, x), i \in 1..h$. We continue the description of this structure in the list of points on the right-hand side below.

– The machine AM is a model describing a set of events modifying the variable x declared in the clause VARIABLES and x is a flexible variable, which allows us to use notations such as x and x'.

– A clause REFINES indicates that the machine AM refines a machine M which is more abstract; however, we will return to this refinement relationship and its role in the development process.

– A particular event defines the initialisation of the variable x according to the before-after predicate $initBAP(s, c, x')$, which characterises the initial values: $Init(s, c, x) \equiv initBAP(s, c, x)$.

– A clause INVARIANTS describes the inductive invariant that this machine is supposed to respect provided that the associated verification conditions are shown to be valid in the theory induced by the context mentioned by the SEES clause.

– The clause THEOREMS introduces the list of safety properties derived in the theory induced by the context and the invariant; these properties relate to the variables and must be proved valid. It is possible to add theorems about sets and constants; this can help the proofs to be made during the verification process.

– To conclude this description, we would like to add that events can carry very important information for the proof process, in particular for proposing witnesses during event refinement. Furthermore, each event has a status (ordinary, convergent, anticipated) which is important in the production of verification conditions. The clause VARIANT is linked to events of convergent status.

We will complete this presentation in the remainder of this section and we will add to it when we describe the refinement. Here is an example of a machine that is important for understanding the difference between an inductive invariant and a safety property.

Example 6 *(abstract machine SAFETY)*

MACHINE SAFETY
VARIABLES x
INVARIANTS
$inv : x = -1$
THEOREMS
$th : x \leq 0$
EVENTS
Event initialisation
begin
$x :| (x' = -1)$
end
Event e
when
$grd : x \geq 0$
then
$act : x :| (x' = x + 1)$
end
END

*The variable x is initialised at -1 and, as a result, the event e is never observed. Since $x = -1$, the invariant inv:$x = -1$ is true and so is the theorem th:$x \leq 0$ which is also true and which is a safety property. This machine observes a system with a state variable that remains constant. The use of Rodin and ProB confirms our point. A reading of the short paper by Van Gasteren and Tel (**?**) clearly reports the difference between inv and th.*

An abstract machine AM is a structure modelling an observed system from the point of view of its state variables but also from the point of view of the domain or domains concerned. Based on the results of section 1.2 and in particular on the definition 12 we will detail the verification conditions produced to ensure the correctness of this machine. These verification conditions will allow us to exploit the example 6. The verification conditions for a system S and its invariance and safety properties are as follows:

– $AX(s, c) \vdash \forall x \in \mathrm{D}.Init(s, c, x) \Rightarrow I(s, c, x)$
– $AX(s, c) \vdash \forall x \in \mathrm{D}.I(s, c, x) \Rightarrow A(s, c, x)$
– For any event e of S,

$$AX(s, c) \vdash \forall x, x' \in \mathrm{D}.I(s, c, x) \wedge BA(e)(s, c, x, x') \Rightarrow I(s, c, x')$$

We have explained that we can simplify these conditions by assuming that x does not occur in $AX(s, c)$, as follows:

– $AX(s,c) \vdash Init(s,c,x) \Rightarrow I(s,c,x)$

– $AX(s,c) \vdash I(s,c,x) \Rightarrow A(s,c,x)$

– For any event e of S,

$$AX(s,c) \vdash I(s,c,x) \wedge BA(e)(s,c,x,x') \Rightarrow I(s,c,x')$$

These verification conditions are translated very directly into a list of more basic verification conditions called *proof obligations*. We will proceed in three steps corresponding to the three conditions. From the AM machine, the following notations are derived:

– s: sets *seen* from the context D .

– c: constants *seen* from the context D .

– x: flexible variables that define the observed state.

– $AX(s,c)$: axioms *seen* from the context D .

– Let $i \in 1..h$. $Th_i(s,c)$: theorems *seen* from the context D and *located* before the theorem th_i; in particular, $Th_1(s,c)$ contains only the theorems of the context.

– $I(s,c,x)$: expression of the invariant of AM defined by $I(s,c,x) \stackrel{def}{=} I_1(s,c,x) \wedge \ldots \wedge I_r(s,c,x)$

– Let $j \in 1..r$. $I_j(s,c,x)$: j-th conjunct of $I(s,c,x)$ of the AM machine.

– $A(s,c,x)$: expression of safety properties of AM defined as $A(s,c,x) \stackrel{def}{=} SAFE_1(s,c,x) \wedge \ldots \wedge SAFE_h(s,c,x)$

– Let $i \in 1..h$. $SAFE_i(s,c,x)$: expression of safety properties.

1.3.2.2. *Proof obligation th/THM*

The verification condition $AX(s,c) \vdash I(s,c,x) \Rightarrow A(s,c,x)$ is reduced to a simplified form $AX(s,c), I(s,c,x) \vdash A(s,c,x)$. Then we apply the rule for introducing the conjunction to produce the following conditions for all $i \in 1..h$, $AX(s,c), I(s,c,x) \vdash SAFE_i(s,c,x)$. The proof can then use properties proved in the previous steps and we have noted them $Th_i(s,c)$ and we can thus add these properties to the hypotheses of the sequent and derive the following verification condition for all $i \in 1..h$, $AX(s,c), Th_i(s,c), I(s,c,x) \vdash SAFE_i(s,c,x)$. The following property is therefore demonstrated.

Propery 5 *(safety)*

If for any $i \in 1..h$, $AX(s,c), Th_i(s,c), I(s,c,x) \vdash SAFE_i(s,c,x)$,
then $AX(s,c) \vdash I(s,c,x) \Rightarrow \bigwedge_{i \in 1..h} SAFE_i(s,c,x)$.

This property is translated into the following verification condition for $i \in 1..h$:

Proof Obligation th$_i$/THM

$AX(s,c), Th_i(s,c), I(s,c,x) \vdash SAFE_i(s,c,x)$

An example of a theorem is the case of the SAFETY machine, which is very simple and has a theorem $th : x \leq 0$ and the verification condition is $x \in \mathbb{Z}, x = -1 \vdash x \leq 0$ which is quite trivially deduced. We will return to this example when we have given the verification conditions that ensure the preservation of the invariant $I(s,c,x)$.

1.3.2.3. *Proof obligation INITIALISATION/inv/INV*

The verification condition $AX(s,c) \vdash Init(s,c,x) \Rightarrow I(s,c,x)$ is reduced to the condition $AX(s,c), Init(s,c,x) \vdash I(s,c,x)$. Finally, we can apply a second reduction transformation by deriving the following conditions: for all $i \in 1..r$, $AX(s,c), x \in \mathrm{D}, Init(s,c,x) \vdash I_i(s,c,x)$. The following property can be deduced.

Propery 6 *(INITIALISATION)*

If for any $i \in 1..r$, $AX(s,c), Init(s,c,x) \vdash I_i(s,c,x)$, *then* $AX(s,c) \vdash Init(s,c,x) \Rightarrow I(s,c,x)$.

Before formulating the verification conditions actually generated by the tool, we recall that the invariant $I(s,c,x)$ is written as a conjunction $I(s,c,x) \equiv \bigwedge_{i \in \{1..r\}} I_i(s,c,x)$. Each element $I_i(s,c,x)$ is labelled $inv_i : I_i(s,c,x)$. This property is translated into the following verification condition for $i \in 1..r$:

Since the initialisation is written $x :| initBAP(s,c,x')$, Rodin states this obligation with the after-values x':

Proof Obligation INITIALISATION/inv$_i$/INV

$AX(s,c), initBAP(s,c,x') \vdash I_i(s,c,x')$

A second check (FIS) is dedicated to the feasibility which is $AX(s,c) \vdash \exists x'.initBAP(s,c,x')$. We assume that the expression $initBAP(s,c,x)$ is written in the action part of the event INITIALISATION in the form $act_1 : x_1 :| initBAP_1(s,c,x'_1), \ldots, act_p : x_p :| initBAP_p(s,c,x'_p)$, with $x = x_1 \ldots x_p$ (x is partitioned as p lists of variables of x) and $initBAP_i(s,c,x'_i)$ is

constructed from $initBAP(s, c, x')$. We therefore assume that $initBAP(s, c, x') \equiv \bigwedge_{i \in \{1..p\}} initBAP_i(s, c, x'_i)$. The decomposition depends on the initial values and one could advise using a normalised form with $p = 1$ but we give the general form. This condition is important to prove as it ensures that the model exists at least in its first state.

Propery 7 *(feasibility of initial conditions)*

Let the following action $act_1 : x_1 : |initBAP_1(s, c, x'_1), \ldots, act_p : x_p : |initBAP_p(s, c, x'_p)$ *defining the conditions for initialising disjoint sub-lists of* $x_1, \ldots x_p$ *variables whose union constitutes the list* x*. The verification condition defined by the sequent* $AX(s, c) \vdash \exists x'.initBAP(s, c, x')$ *is equivalent to the list of sequents* $AX(s, c) \vdash \exists x'_i.initBAP_i(s, c, x'_i)$ *for* $i \in \{1..p\}$*.*

Proof Obligation INITIALISATION/act$_i$/FIS

For any $i \in \{1..p\}$, $AX(s, c) \vdash \exists x'_i.initBAP_i(s, c, x'_i)$.

Example 7 *(feasibility for the SAFETY machine)*

In the example SAFETY, the initialization is established, by proving $\vdash \exists x'.x' = -1$ *which is in fact trivially true for* $x' = -1$*.*

Example 8 *(feasibility with a witness) We can also initialize a variable* acc *that must satisfy an invariant of the type* $acc \in P \rightarrow B \wedge acc \subseteq a$ *where* a *is a constant modelling a table of access authorisations for people* (P) *to buildings* (B)*, i.e.* $a \subseteq P \times B$*. The initialisation event is written very simply in the form* $acc : |(acc' \in P \rightarrow B \wedge acc' \subseteq a)$ *which expresses the fact that the initial value of* acc *must satisfy the invariant but does not explicitly give a value* $acc0$*. The verification condition is then obtained as follows:* $a \subseteq P \times B \vdash \exists acc'.acc' \in P \rightarrow B \wedge acc' \subseteq a$*. The solution is to declare a constant* $acc0$*, with the axiom* $acc0 \in P \rightarrow B \wedge acc0 \subseteq a$*, which can be used as a witness in the proof. This only moves the difficulty to the context: the axiom on* $acc0$ *is consistent only if* a *contains a total function from* P *to* B*, that is, if every person is authorised to access at least one building (see the warning on the consistency of* $AX(s, c)$*).*

Example 9 *(example of an assignment)*

A final example is the assignment of an expression e *to* x*, i.e.* $x := e$*. The feasibility condition* $\exists x'.\ x' = e$ *is trivially true (take* $x' = e$*), so no FIS obligation is generated; only the well-definedness of* e *remains to be proved, by the obligation act/WD.*

The Rodin platform generates an obligation FIS only for the *non-deterministic* actions, $x :\in E$ and $x : |P(x, x')$; for $x :\in E$ the obligation reduces to $E \neq \varnothing$. A deterministic action $x := e$ produces only the obligation WD. Rodin also splits feasibility action by action (act_i/FIS); this is sound because the actions of an event modify disjoint lists of variables, so that the conjunction of their before-after predicates is feasible if, and only if, each of them is. The price is that the feasibility of an action is proved in isolation: a non-deterministic action whose before-after predicate is contradictory only together with the guard is detected, but the obligation gives no hint on how to repair the model, by strengthening the guard or weakening the action.

1.3.2.4. *Proof obligations e/inv/INV and e/act/FIS*

In the definition 14, we give these two verification conditions which are intended to ensure the preservation of the property $I(s, c, x)$ but also to ensure the feasibility of the event e when the invariant $I(s, c, x)$ is valid. The expressions for these verification conditions are given below:

$$\begin{cases} (FIS)\ AX(s,c), I(s,c,x), G(u,s,c,x) \vdash \exists x'.BAP(u,s,c,x,x') \\ (INV)\ AX(s,c), I(s,c,x), G(u,s,c,x), BAP(u,s,c,x,x') \vdash I(s,c,x') \end{cases}$$

We give an initial example which illustrates the central role of these conditions. We take the SAFETY machine and the event e defined by Event e when grd : $x \geq 0$ then $act : x : |(x' = x + 1)$ end and we consider the invariant $I(x) \stackrel{def}{=} inv : x = -1$. e/inv/INV is defined by the following expression: $I(x), x \geq 0, x' = x + 1 \vdash I(x')$ and which is reduced to this expression $x = -1, x \geq 0, x' = x + 1 \vdash x' = -1$ which is trivially true since the hypotheses are inconsistent. A first attempt might have been to replace this invariant by $J(x) \stackrel{def}{=} x \leq 0$ and, in this case, the expression to be proved would have been $J(x), x \geq 0, x' = x + 1 \vdash J(x')$ which simplifies to $x \leq 0, x \geq 0, x' = x + 1 \vdash x' \leq 0$, then to $x \leq 0, x \geq 0, x = 0, x' = x + 1 \vdash 1 \leq 0$! Obviously, it should be stressed that $I(x)$ is an inductive invariant and that $J(x)$ is a weaker invariance property which we will call a safety property or a theorem. So our SAFETY machine is valid since we can prove that $I(x) \vdash J(x)$. This example shows the difference between an inductive invariance property ($x = -1$) and an invariance property or always true or safety property ($x \leq 0$); this difference was pointed out by A. J. M. van Gasteren and G. Tel (**?**). The feasibility condition is obvious and therefore poses no particular problem. We will now describe the verification conditions generated by the Rodin tool and the designation of the verification conditions. A property of the sequent calculus is used to break the verification conditions and thus divide the proof effort.

Propery 8 *(introduction/Elimination of connector $\wedge$)*

Consider the sequent $\Gamma \vdash \bigwedge_{i \in \{1..t\}} P_i$.

The proof of this sequent amounts to proving the sequents $\Gamma \vdash P_i$*, for all indices* $i \in \{1..t\}$.

We apply this property to the case of the invariant $I(s,c,x)$ equivalent to $\bigwedge_{i\in\{1..r\}} I_i(s,c,x)$ and with the labelling of each $I_i(s,c,x)$ assigning it the label inv_i. This property is translated into the following verification condition for $i \in 1..r$:

Proof Obligation e/inv$_i$/INV

$$AX(s,c), I(s,c,x), G(u,s,c,x), BAP(u,s,c,x,x') \vdash I_i(s,c,x')$$

A second check (FIS) is dedicated to the feasibility of the event e, which is $AX(s,c), I(s,c,x), G(u,s,c,x) \vdash \exists x'.BAP(u,s,c,x,x')$. As for the initialisation, Rodin states it action by action, for $act_i : x_i : |BAP_i(u,s,c,x,x'_i)$:

Proof Obligation e/act$_i$/FIS

$$AX(s,c), I(s,c,x), G(u,s,c,x) \vdash \exists x'_i.BAP_i(u,s,c,x,x'_i).$$

1.3.2.5. *Proof obligations e/act/WD and INITIALISATION/act/WD*

The verification conditions (INV) and (FIS) ensure that the invariant is preserved and that the events are feasible. However, other conditions remain to be verified in order to guarantee the validity of formalised objects. The division by zero is an element which testifies to this need to avoid the *silly* expressions mentioned by L. Lamport (**??**). Writing of formal expressions is therefore perilous and must be framed by the verification conditions WD applied for guards and for actions.

Proof Obligation e/grd$_i$/WD

$$AX(s,c), I(s,c,x), G_1(u,s,c,x), \ldots, G_{i-1}(u,s,c,x) \vdash WD(G_i(u,s,c,x)).$$

The guards that precede G_i are hypotheses of this obligation: the list of guards is read as the conjunction $G_1 \wedge \ldots \wedge G_k$, and the rule $WD(P \wedge Q) = WD(P) \wedge (P \Rightarrow WD(Q))$ of Table 1.1 is applied from left to right. This makes it possible to write, for instance, the two guards $grd_1 : x \in dom(f)$ and $grd_2 : f(x) > 0$: the obligation for grd_2 has $x \in dom(f)$ as a hypothesis.

Rodin applies the same left-to-right reading to every labelled list: the well-definedness of an axiom, an invariant or a guard is proved with the axioms,

invariants or guards that precede it, and the well-definedness of an action with all the guards. The order of the labelled predicates therefore matters: exchanging grd_1 and grd_2 above yields the obligation $AX(s,c), I(s,c,x) \vdash x \in dom(f)$, which is in general not provable. Within a single predicate, the conjunction is not commutative for well-definedness either: $x \in dom(f) \wedge f(x) > 0$ is well defined, whereas $f(x) > 0 \wedge x \in dom(f)$ is not. Rodin discharges most WD obligations automatically, so that an undischarged WD obligation usually points to a missing or misplaced hypothesis rather than to a difficult proof.

Proof Obligation e/act$_i$/WD

$$AX(s,c), I(s,c,x), G(u,s,c,x) \vdash WD(BAP_i(u,s,c,x,x_i')).$$

Proof Obligation INITIALISATION/act$_i$/WD

$$AX(s,c) \vdash WD(initBAP_i(s,c,x_i'))$$

We have presented the three kinds of verification conditions: WD, INV and FIS for validating the inductive property of the assertion $I(s,c,x)$. We are continuing our gradual discovery of the elements that make it possible to model a set of events.

1.3.2.6. *Status of an event*

An event e in Event-B is defined by its parameters u, its guards $G(u,s,c,x)$ and its actions $x : |BAP(u,s,c,x,x')$ but it has one of three possible *status ordinary*, *convergent* or *anticipated*. In principle, an Event-B machine preserves an invariant and safety properties called theorems. However, it is possible to use a *variant*, an expression whose value is a natural number or a finite set and which must decrease, to prove the convergence of some events. To understand this point, we need to remember that the induction rules relating to termination are expressed by a sequence of assertions $P_n(x)$ such that for each $n \in \mathbb{N}$, P_{n+1} *leads to* P_n by the observation of at least one *convergent* event. In fact, the problem is to show that the property $\exists n \in \mathbb{N}.P_n(x)$ *leads* to a property $Q(x)$ which is linked by the implication $P_0(x) \Rightarrow Q(x)$. Readers familiar with temporal logics will have recognised a property of liveness expressed with the operator *leads to* denoted $\leadsto$ (**??????**). Thus, a liveness property of the form $P(x) \leadsto Q(x)$ requires the use of a well-founded induction rule, as in UNITY or TLA, such as:

IF

– $I(x)$ is the invariant of the current machine,

– $R_n(x)$ is a sequence of assertions satisfying the property $R_{n+1}(x) \leadsto R_n(x)$, for all $n \in \mathbb{N}$,

– $P(x) \wedge I(x) \Rightarrow \exists n \in \mathbb{N}.R_n(x)$

– $R_0(x) \Rightarrow Q(x)$

THEN $P(x) \leadsto Q(x)$.

In the case of our machine, the sequence $R_n(x)$ is defined by $R_n(x) \stackrel{def}{=} I(x) \wedge variant \leq n$. In fact, we find the Floyd-Hoare method and a way of showing that a machine converges to a stable state, if the variant decreases each time a convergent event is observed. The decrease of the variant only guarantees that the convergent events cannot occur forever; to conclude $P(x) \leadsto Q(x)$, a convergent event must also remain enabled as long as $Q(x)$ does not hold, which follows from deadlock freedom or from a fairness assumption. We will now define the verification conditions associated with the proof of convergence.

The variant $var : varexp(s, c, x)$ is assumed to be defined for the current machine. Two conditions are required:

– The variant $var : varexp(s, c, x)$ is defined for any convergent event: $AX(s, c), I(s, c, x), G_e(u, s, c, x) \vdash varexp(s, c, x) \in \mathbb{N}$ (NAT); for a set-valued variant, the condition is $AX(s, c), I(s, c, x), G_e(u, s, c, x) \vdash \text{finite}(varexp(s, c, x))$ (FIN)

– The variant $var : varexp(s, c, x)$ varies for any convergent event: $AX(s, c), I(s, c, x), G_e(u, s, c, x), BAP(u, s, c, x, x') \vdash varexp(s, c, x') < varexp(s, c, x)$ (VAR)

For a set-valued variant, we have the verification condition $AX(s, c), I(s, c, x), G_e(u, s, c, x), BAP(u, s, c, x, x') \vdash varexp(s, c, x') \subset varexp(s, c, x)$ (VAR). To sum up, we have two new verification conditions produced when there are convergent events and variants.

Proof Obligation e/NAT

$AX(s, c), I(s, c, x), G_e(u, s, c, x) \vdash varexp(s, c, x) \in \mathbb{N}$

Proof Obligation e/VAR (numeric variant)

$AX(s, c), I(s, c, x), G_e(u, s, c, x), BAP(u, s, c, x, x') \vdash varexp(s, c, x') < varexp(s, c, x)$

Proof Obligation e/VAR (set-valued variant)

$AX(s,c), I(s,c,x), G_e(u,s,c,x), BAP(u,s,c,x,x') \vdash varexp(s,c,x') \subset varexp(s,c,x)$

We have devised a small example Fig. 1.1 which brings together the verification conditions with variant.

Example 10 *(modelling an adder)*

Consider two events evt2 *and* evt3 *which decrease by one unit respectively the variables x and y initialised respectively at $x0$ and $y0$ two natural numbers. These two events converge and help to reach a state where the two variables x and y are zero, leaving only the event* evt1 *observable. Each time an event decreases x or y, it increases z by one unit. Subject to an implicit assumption of weak global fairness over the events, we can guarantee that z will be equal to the sum $x0 + y0$. This example introduces the notion of a convergent event, and it remains to comment on the status of event anticipated to give all the possible forms of events in a basic machine. Thus, the event* evt4 *is a form of anticipation of what could be added later in this machine. In fact, we are going to present the refinement which will enable us to add and specify elements due to the basic machine. In this case, we could imagine adding events which compute more quickly but which must respect and not disturb convergence.*

1.4. Refinement of Event-B machines

The refinement relation is a mechanism which allows models (Event-B machines) to be developed in an incremental and progressive way, starting from a very abstract model and, following a chain of refinements, reaching a concrete expression of the system model. The sequence of machines is an exercise combining modelling and proof. The main idea of refinement is to diffuse the proof effort and to allow a description on several levels of abstraction of a system to be modelled. For example, a communication protocol can be modelled by a one-step communication action, expressing only the expected service. Then we can give details of the more elementary actions which contribute to this service and then we can propose a model which makes it possible to localize the actions. The modelling of the IEEE 1394 protocol (**?**) is an example of development by refinement of a distributed algorithm which is seen as a transformation of a forest into a tree using important properties of a connected undirected graph. The chapter entitled *Correct by Construction Communication Models for Distributed Algorithms* is devoted to the incremental development of distributed systems. The diagram below describes the general framework of the use of refinement: the machine

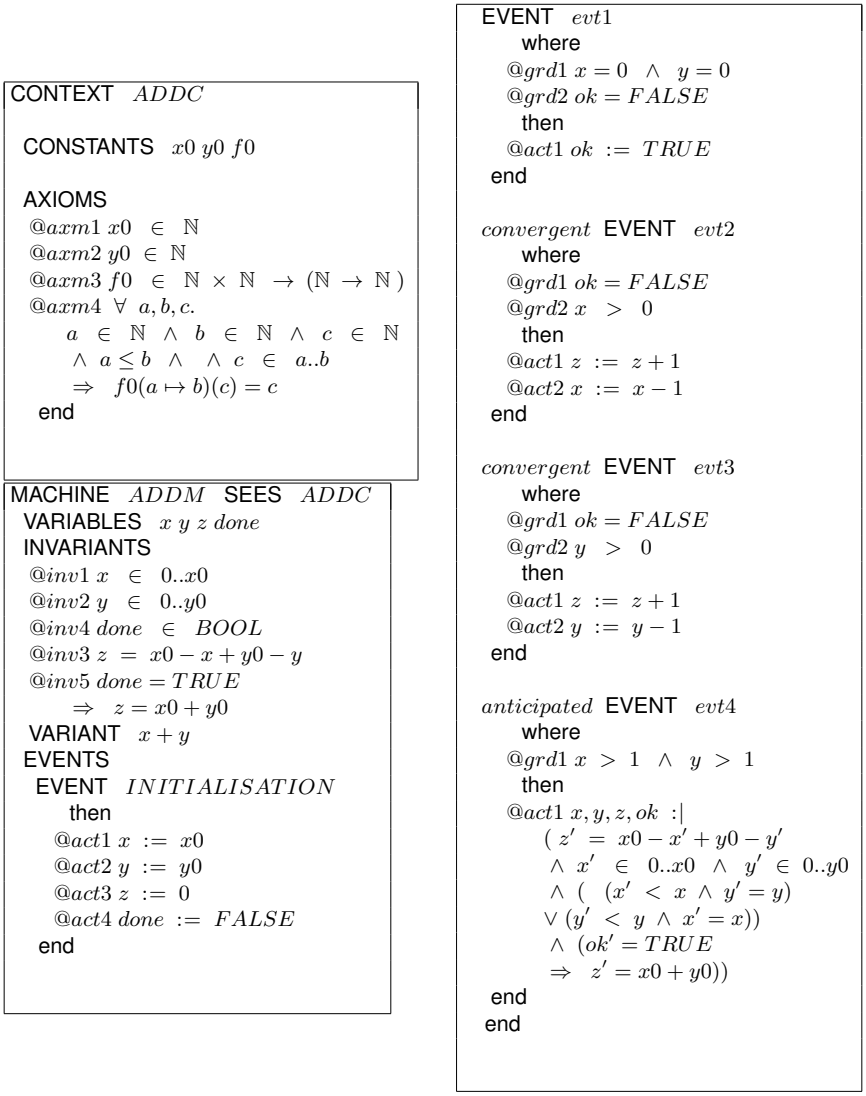
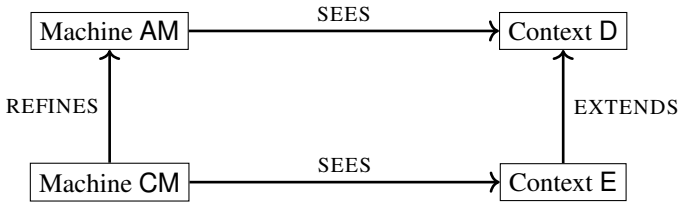


Figure 1.1. *Example of convergent and anticipated events*

CM refines AM, and the context E , seen by CM, extends the initial context D . We will explain some general properties of refinement and then present the verification conditions for the refinement. In the subsection 1.3.2.6, we described the various statuses of an event in an abstract machine AM without linking this concept to refinement. We will provide methodological elements that justify these statuses.



1.4.1. *Elements on the refinement*

The refinement relationship can refer to several aspects of the relationship between models of the same system. To some extent, a model M1 refines a model M2, if M1 is more precise than M2 and if it gives more information about the system being modelled. This enrichment of the current model must not call into question what has already been acquired. The refinement of Event-B machines is a practice that makes it possible to make an abstract model closer to the observed system. It proceeds by successive and increasingly precise observations, by detailing elements in the concrete model. Refinement defines a series of levels of increasingly concrete observations. The definition is based on the refinement of events and on the definition of a glueing invariant establishing the relationship between the variables of the abstract Event-B machine and the concrete variables of the concrete Event-B machine. Before defining Event-B refinement, we recall some elements linking the evolution of B to Event-B . During the first B conference (**?**) organised by H. Habrias in 1996, J.-R. Abrial (**?**) gave a demonstration of the transition from B to Event-B while preserving the tools implemented in Atelier B. A B machine contains operations and an Event-B machine contains events. Michael Leuschel (**?**) has written a detailed document outlining the differences between the B notation and the Event-B notation. Our comparison does not take into account the notations but the intention of the events. The difference between an operation and an event is important when modelling a system and must be used with care and precision. Jean-Raymond Abrial distinguishes between two types of model: *abstract machines*, which offer operations called by their environment, and *abstract systems*, which observe events that occur spontaneously when their guards hold (**?**). We take up the initial definitions of event refinement and describe the various verification conditions that follow from them. We give the initial definition of the refinement of two events expressed in the context of the B language.

Definition 15 *(refinement between two events (I))*

Let x be the abstract variable (or list of variables) and $I(s, c, x)$ the abstract invariant, y the concrete variable (or list of variables) and $J(s, c, x, y)$ the gluing invariant.

Let c *be a concrete event observing the variable y and* a *an event observing the variable x and preserving $I(s, c, x)$.*

Event c *refines event* a *with respect to x, $I(s, c, x)$, y and $J(s, c, x, y)$, if*

$$AX(s, c) \vdash I(s, c, x) \wedge J(s, c, x, y) \Rightarrow [\mathsf{c}]\, \neg[\mathsf{a}]\, \neg J(s, c, x, y).$$

Definition 15 is exactly the definition of the refinement of *operations* in the B language (**?**), where operations are replaced by *events*. An event is observed whereas an operation is called, but the refinement relation is the same and is expressed in the wp calculus. The events are written in sans-serif type, a and c, to distinguish the concrete event c from the list c of constants. The two events are defined as follows:

$$\mathsf{a} \stackrel{def}{=} \left\{ \begin{array}{l} \textsf{any } u \textsf{ where} \\ \quad G(u, s, c, x) \\ \textsf{then} \\ \quad x : |ABAP(u, s, c, x, x') \\ \textsf{end} \end{array} \right. \qquad \mathsf{c} \stackrel{def}{=} \left\{ \begin{array}{l} \textsf{any } v \textsf{ where} \\ \quad H(v, s, c, y) \\ \textsf{witness} \\ \quad u : WP(u, s, c, v, y) \\ \quad x' : WV(v, s, c, y', x') \\ \textsf{then} \\ \quad y : |CBAP(v, s, c, y, y') \\ \textsf{end} \end{array} \right.$$

The events a and c are equivalent to the following normalised forms, which use their before-after relations $BA(\mathsf{a})$ and $BA(\mathsf{c})$:

– a is equivalent to begin $x : |(\exists u.G(u, s, c, x) \wedge ABAP(u, s, c, x, x'))$ end;

– c is equivalent to begin $y : |(\exists v.H(v, s, c, y) \wedge CBAP(v, s, c, y, y'))$ end.

The witnesses $u : WP(u, s, c, v, y)$ (*witness for a parameter*) and $x' : WV(v, s, c, y', x')$ (*witness for a variable*) are hints for the proof tool: they tell it how to instantiate an existential quantifier. In a sense, they also refer to the abstract model: u and x' no longer exist in the concrete machine, and the witnesses express them in terms of the concrete state. Witnesses are required in some cases: for every abstract parameter that disappears, and for every abstract variable that disappears and is modified non-deterministically by the abstract event; elsewhere, they are optional but they help the proof tools. We first transform Definition 15 without witnesses.

Explanation 2 *(characterisation of refinement)*

We abbreviate $\Gamma \stackrel{def}{=} AX(s, c), I(s, c, x), J(s, c, x, y)$.

(1) $\Gamma \vdash [\mathsf{c}]\, \neg[\mathsf{a}]\, \neg J(s,c,x,y)$ *(Definition 15, after moving* $I \wedge J$ *to the hypotheses)*

is equivalent to (definition of $[\mathsf{a}]$*:* $[\mathsf{a}]\, \neg J(s,c,x,y) \equiv \forall x'.\, \big((\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x')) \Rightarrow \neg J(s,c,x',y)\big)$*)*

(2) $\Gamma \vdash [\mathsf{c}]\, \neg \forall x'.\, \big((\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x')) \Rightarrow \neg J(s,c,x',y)\big)$

is equivalent to ($\neg \forall x'.(A \Rightarrow \neg B) \equiv \exists x'.(A \wedge B)$*)*

(3) $\Gamma \vdash [\mathsf{c}]\, \exists x'.\, \big((\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x')) \wedge J(s,c,x',y)\big)$

is equivalent to (definition of $[\mathsf{c}]$*:* $[\mathsf{c}]R \equiv \forall y'.\big((\exists v.H(v,s,c,y) \wedge CBAP(v,s,c,y,y')) \Rightarrow R[y'/y]\big)$*, since* v *is not free in* R*)*

(4) $\Gamma \vdash \forall y'.\, \Big((\exists v.H(v,s,c,y) \wedge CBAP(v,s,c,y,y')) \Rightarrow \exists x'.\, \big((\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x')) \wedge J(s,c,x',y')\big)\Big)$

is equivalent to ($\forall$*-introduction on the right:* y' *is not free in* Γ*)*

(5) $\Gamma \vdash (\exists v.H(v,s,c,y) \wedge CBAP(v,s,c,y,y')) \Rightarrow \exists x'.\, \big((\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x')) \wedge J(s,c,x',y')\big)$

is equivalent to ($\Gamma \vdash A \Rightarrow B$ *is equivalent to* $\Gamma, A \vdash B$*)*

(6) $\Gamma,\ \exists v.H(v,s,c,y) \wedge CBAP(v,s,c,y,y') \vdash \exists x'.\, \big((\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x')) \wedge J(s,c,x',y')\big)$

is equivalent to ($\exists$*-elimination on the left:* v *is not free in the conclusion)*

(7) $\Gamma,\ H(v,s,c,y),\ CBAP(v,s,c,y,y') \vdash \exists x'.\, \exists u.\, \big(G(u,s,c,x) \wedge ABAP(u,s,c,x,x') \wedge J(s,c,x',y')\big)$

implies, but is not equivalent to (weakening of the conclusion)

(8)

1) $\Gamma,\ H(v,s,c,y),\ CBAP(v,s,c,y,y') \vdash \exists u.\, G(u,s,c,x)$

2) $\Gamma,\ H(v,s,c,y),\ CBAP(v,s,c,y,y') \vdash \exists x'.\, \exists u.\, \big(ABAP(u,s,c,x,x') \wedge J(s,c,x',y')\big)$

Steps (1) to (7) are equivalences. Step (8) is only an implication: (7) requires *one* value of u that both satisfies the abstract guard and produces the abstract step,

whereas (8a) and (8b) may use *two different* values of u. For instance, take $u \in \{0,1\}$, $G(u,s,c,x) \equiv u = 0$, $ABAP(u,s,c,x,x') \equiv u = 1 \wedge x' = x+1$ and $J \equiv true$: (8a) holds with $u = 0$ and (8b) holds with $u = 1$, but no value of u satisfies both G and $ABAP$, so (7) is false. Step (8) therefore yields necessary conditions, guard strengthening (GRD) and simulation (SIM), which are not sufficient on their own.

Propery 9 *(refinement between events (II))*

Let x, $I(s,c,x)$, y, $J(s,c,x,y)$, a and c be as in Definition 15. Event c refines event a with respect to x, $I(s,c,x)$, y and $J(s,c,x,y)$ if, and only if,

$$\Gamma,\ H(v,s,c,y),\ CBAP(v,s,c,y,y') \vdash \exists x'.\, \exists u.\, \big(G(u,s,c,x) \wedge ABAP(u,s,c,x,x') \wedge J(s,c,x',y')\big).$$

Consequently, if event c refines event a, then

1) (GRD) $AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,y) \vdash \exists u.G(u,s,c,x)$*;*

2) (SIM) $\begin{cases} AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,y), CBAP(v,s,c,y,y') \\ \vdash \exists x'.\, \exists u.\, \big(ABAP(u,s,c,x,x') \wedge J(s,c,x',y')\big). \end{cases}$

The converse of this last statement does not hold in general.

In GRD, the hypothesis $CBAP(v,s,c,y,y')$ of (8a) has been dropped: the abstract guard does not depend on y', and this is the form used by Rodin. Dropping a hypothesis makes the condition stronger, so GRD still implies (8a).

The witnesses restore the link between the two conditions. They rely on the following sequent rule, in which u is a variable that is not free in Γ:

$$\text{if} \quad \Gamma, W(u) \vdash P(u) \quad \text{and} \quad \Gamma \vdash \exists u.W(u), \quad \text{then} \quad \Gamma \vdash \exists u.P(u).$$

The rule is sound: $\exists$-introduction on the right turns the first premise into $\Gamma, W(u) \vdash \exists u.P(u)$; since u is not free in Γ nor in the conclusion, $\exists$-elimination on the left gives $\Gamma, \exists u.W(u) \vdash \exists u.P(u)$; a cut with the second premise concludes. The second premise is essential: with $W \equiv false$, the first premise holds for every P.

Propery 10 *(refinement between events with witnesses (III))*

Let x, $I(s,c,x)$, y, $J(s,c,x,y)$, a and c be as in Definition 15, and let $\Gamma \stackrel{def}{=} AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,y), CBAP(v,s,c,y,y')$. If the following sequents are valid:

1) (GRD-WIT)
$AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,y), WP(u,s,c,v,y) \vdash G(u,s,c,x)$;
2) (SIM-WIT)
$\Gamma, WP(u,s,c,v,y), WV(v,s,c,y',x') \vdash ABAP(u,s,c,x,x') \wedge J(s,c,x',y')$;
3) (WFIS-P)
$AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,y) \vdash \exists u.WP(u,s,c,v,y)$;
4) (WFIS-V)
$\Gamma \vdash \exists x'.WV(v,s,c,y',x')$;

then event c *refines event* a *with respect to* x, $I(s,c,x)$, y *and* $J(s,c,x,y)$.

Explanation 3 *(proof of Property 10) By the equivalence of (1) and (7), it suffices to prove (7). Assume* Γ. *By* WFIS-P, *there is a value* u_0 *such that* $WP(u_0,s,c,v,y)$, *and by* WFIS-V *there is a value* x'_0 *such that* $WV(v,s,c,y',x'_0)$. GRD-WIT, *instantiated with* u_0, *gives* $G(u_0,s,c,x)$. SIM-WIT, *instantiated with* u_0 *and* x'_0, *gives* $ABAP(u_0,s,c,x,x'_0) \wedge J(s,c,x'_0,y')$. *Hence* $G(u_0,s,c,x) \wedge ABAP(u_0,s,c,x,x'_0) \wedge J(s,c,x'_0,y')$, *which is the conclusion of (7) with* $u := u_0$ *and* $x' := x'_0$.

The key point is that GRD-WIT and SIM-WIT share the hypothesis $WP(u,s,c,v,y)$: the same value of u is used to prove the abstract guard and to simulate the abstract action. This is exactly what step (8) loses. The witnesses are therefore not merely an aid for the prover: they make the decomposition into separate proof obligations sound, and the feasibility conditions WFIS-P and WFIS-V are required for this soundness.

When a witness is deterministic, for instance $u = e(s,c,v,y)$ or $x' = f(s,c,y')$, the one-point rule replaces the variable by the expression and the corresponding WFIS condition holds trivially; Rodin generates WFIS only for non-deterministic witnesses, and a well-definedness obligation WWD for every witness (Section 1.4.3.2). A common choice for the variable witness is $WV \equiv J(s,c,x',y')$: the invariant part of SIM-WIT then becomes trivial, and WFIS-V states that every concrete state has an abstract counterpart.

Finally, Definition 15 does not ensure that c is feasible: a concrete event whose guard holds but whose action yields no after-state satisfies $[\mathsf{c}]R$ for every R. The feasibility of c must therefore be proved separately, by the proof obligation c/act/FIS.

We have omitted the verification condition corresponding to the initialization and it is simply obtained by the same transformations as before. We summarize the list of proof obligations required for checking refinement of Event-B models.

Propery 11 *(verification condition for refinement initialisation)*

Let $AInit(s, c, x')$ *and* $CInit(s, c, y')$ *be the initialization predicates for the abstract machine and the concrete machine respectively. The refinement condition for initialization is an adaptation of the refinement relation:*

(INIT) $AX(s, c), CInit(s, c, y') \vdash \exists x'.AInit(s, c, x') \wedge J(s, c, x', y')$

In order to simplify the conditions stated as sequents, we apply two very simple rules for calculating sequents.

Propery 12 *(simplification of sequents)*

1) $AX(s, c), H \vdash P_1 \Rightarrow P_2 \Rightarrow \ldots \Rightarrow P_n \Rightarrow Q$ *is equivalent to* $AX(s, c), H, P_1, P_2, \ldots, P_n \vdash Q$

2) $AX(s, c), H \vdash P_1 \wedge P_2 \wedge \ldots \wedge P_n$ *is equivalent to*

a) $AX(s, c), H \vdash P_1$

b) $\ldots$

c) $AX(s, c), H \vdash P_n$

By applying these two rules to our various verification conditions, we obtain the list of verification conditions expressed by sequents. We give a list of the verification conditions to be produced and verified to ensure the refinement of the abstract machine by the concrete machine. The two events are related by the refinement relationship.

Propery 13 *(proof obligations for Event-B refinement)*

– *(INIT)* $AX(s, c), CInit(s, c, y') \vdash \exists x'.AInit(s, c, x') \wedge J(s, c, x', y')$

– *(GRD)* $\begin{cases} AX(s, c), \\ I(s, c, x), J(s, c, x, y), H(v, s, c, y) \\ \vdash \\ \exists u.G(u, s, c, x) \end{cases}$

– *(GRD-WIT)* $\begin{cases} AX(s, c), \\ WP(u, s, c, v, y), \\ I(s, c, x), J(s, c, x, y), H(v, s, c, y) \\ \vdash \\ G(u, s, c, x) \end{cases}$

– *(SIM)* $\begin{cases} AX(s, c), I(s, c, x), J(s, c, x, y), H(v, s, c, y), CBAP(v, s, c, y, y') \\ \vdash \\ \exists x'.\exists u.ABAP(u, s, c, x, x') \wedge J(s, c, x', y') \end{cases}$

– *(SIM-WIT)* $\begin{cases} AX(s,c), \\ I(s,c,x), J(s,c,x,y), H(v,s,c,y), CBAP(v,s,c,y,y'), \\ WP(u,s,c,v,y), WV(v,s,c,y',x') \\ \vdash \\ ABAP(u,s,c,x,x') \wedge J(s,c,x',y') \end{cases}$

– *(WFIS-P)* $\begin{cases} AX(s,c), \\ I(s,c,x), J(s,c,x,y), H(v,s,c,y) \\ \vdash \\ \exists u.WP(u,s,c,v,y) \end{cases}$

– *(WFIS-V)* $\begin{cases} AX(s,c), \\ I(s,c,x), J(s,c,x,y), H(v,s,c,y), CBAP(v,s,c,y,y') \\ \vdash \\ \exists x'.WV(v,s,c,y',x') \end{cases}$

– *(FIS)* $AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,y) \vdash \exists y'.CBAP(v,s,c,y,y')$

– *(TH)* $AX(s,c), Th(s,c), I(s,c,x), J(s,c,x,y) \vdash SAFE(s,c,x,y)$

This list is not to be proved as a whole: without witnesses, GRD and SIM are generated; with witnesses, Rodin generates GRD and SIM with the witness predicates as hypotheses, together with WFIS for the non-deterministic witnesses and WWD for all witnesses. The suffixes -WIT, -P and -V are ours.

Now, we describe the concept of refinement in the language itself and illustrate these conditions, to show their role and impact on correct development by construction. From a methodological and proof-theoretical point of view, the explicit witnesses make the decomposition into separate proof obligations sound and simplify the proofs.

1.4.2. *Refinement machines in Event-B*

1.4.2.1. *The structure of refinement machine*

A basic abstract machine is described in figure 1.1 and does not include a reference to a more abstract machine. The refines clause did not appear. The abstract machine AM in subsection 1.3.2.1 mentions a machine M that is refined by this same machine AM. We establish a refinement relationship between a machine denoted CM and another machine denoted AM.

The machine CM (see Fig:1.2) is a model describing a set of events E(CM) modifying the variable y declared in the clause VARIABLES. A clause REFINES indicates that the CM machine refines a AM machine and E(AM) is the set of abstract events in AM. A particular event defines the initialisation of variable y according to the relationship $CInit(s,c,y')$. The property "Event c refines event a with respect to x, $I(s,c,x)$, y

MACHINE CM
REFINES AM
SEES E
VARIABLES y
INVARIANTS
$inv_1 : J_1(s, c, x, y)$
...
$inv_r : J_r(s, c, x, y)$
THEOREMS
$th_1 : SAFE_1(s, c, x, y)$
...
$th_h : SAFE_h(s, c, x, y)$
VARIANT
$var : varexp(s, c, y)$
EVENTS
Event initialisation
begin
$y :| (CInit(s, c, y'))$
end
...
Event c
refines a
any v where
$H(v, s, c, y)$
witness
$u : WP(u, s, c, v, y)$
$x' : WV(v, s, c, y', x')$
then
$y :| CBAP(v, s, c, y, y')$
end
...
END

Figure 1.2. *Refinement machine* CM

and $J(s, c, x, y)$" is denoted by the expression c refines a. Events a and c are attached to two machines AM and CM; the invariant attached to each event is the invariant of its machine. A clause INVARIANTS describes the inductive invariant $J(s, c, x, y)$ that this machine is assumed to respect provided that the associated verification conditions are shown to be valid in the theory induced by the context E mentioned by the clause SEES. $J(s, c, x, y)$ is the gluing invariant linking the variable y to the variable x. The clause THEOREMS introduces the list of safety properties derived in the theory. These properties relate to the variables y and x and must be proved valid. It is possible to add theorems about sets and constants; this can help the proofs to be carried out during the verification process. To conclude this description, we would like to add that events can carry very important information for the proof process, in particular for proposing witnesses during event refinement. Furthermore, each event has a status (ordinary, convergent, anticipated) which is important in the production of verification conditions. The clause VARIANT is linked to events of convergent and anticipated status. The event c (concrete) explicitly refines an event a of the AM machine.

Figure 1.3 shows the abstract and concrete traces, linked by the gluing invariant J; the organisation of machines and contexts according to the relationships SEES, EXTENDS and REFINES is shown at the beginning of Section 1.4. We have defined the refinement relation for two events a and c and we can propose an extension for machines. We denote refines the refinement relation defined in the previous section for events and it is possible that several concrete events in CM refine an abstract event a and it is possible to have a concrete event e which is refining several abstract events and it will be a special case called *merging of abstract events*.

Definition 16 *(Event-B machine refinement)*

The machine CM *refines the machine* AM*, if the concrete initialisation refines the abstract one (condition* INIT*) and if any event* c *of* CM *refines an event* a *of* AM*, the new events refining* skip*:*

$$\forall \mathsf{c}.\mathsf{c} \in \mathsf{E}(\mathsf{CM}) \Rightarrow \exists \mathsf{a}.\mathsf{a} \in \mathsf{E}(\mathsf{AM}) \wedge \mathsf{c} \text{ refines } \mathsf{a}.$$

Each machine has an event skip which does not modify the machine's variables. A concrete event c can refine an event skip whose effect is not to modify x in the abstract machine AM. We assume that the invariant of AM is $I(s, c, x)$ and that the initialisation of AM is $AInit(s, c, x')$. The philosophy of incremental modelling is based on the need to support proofs, and requires modelling to be carried out in conjunction with proofs. The proof witnesses are used to give properties of the parameter u and the variable x which have disappeared in the machine CM but for which the user must give an expression according to the state of CM. In Figure 1.3, the schematisation of the refinement relationship shows what is gained. Indeed, $I(s, c, x)$ is not *reproved* but is preserved insofar as the event c does not invalidate $I(s, c, x)$ at the next step.

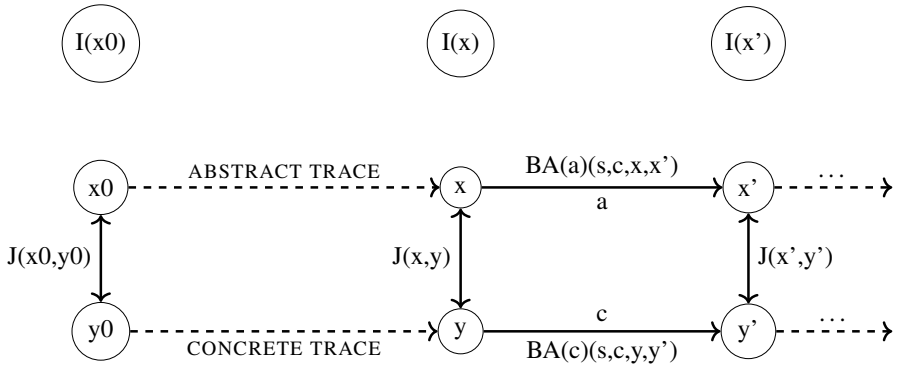


Figure 1.3. *Refinement between two machines*

We give a very simple example which shows what the refinement of two machines can express.

Example 11 *(example of clock)*

A machine M1 reports observations of hours and a machine M2 reports hours and minutes. These machines are described in figure 1.4. This is a very special case of refinement called superposition and the proof is fairly straightforward. The event skip is explicitly added in our text but it is left implicit in the Rodin archive.

We will now use the verification conditions set out in the property 13 to give the identifications of the verification conditions as they appear in the Rodin tool.

1.4.3. *Proof obligations for refinement*

The verification conditions for refinement are identical to those we have already presented, with one important difference: they use abstract elements such as the abstract invariant and the abstract event. Simplification of the conditions is enhanced by reference to explicit witnesses. Finally, the gluing invariant $J(s, c, x, y)$ states a relationship between the abstract variables x and the concrete variables y, and these two expressions in fact designate a list of variables. It is possible to simplify the verification conditions when there is a variable z in the abstract machine AM which is declared in the concrete machine CM in the form z. The translation must take the context into account and we could use az for AM and cz for CM. The link between the two variables would then be explained by the relationship $az = cz$. The choice is to keep the expression z which is both an occurrence of an abstract variable and an occurrence

CONTEXT C

CONSTANTS H M

AXIOMS
$@axm1$ $H = 0..23$
$@axm2$ $M = 0..59$
end

MACHINE $M1$ SEES C

VARIABLES h

INVARIANTS
$@inv1$ $h \in H$

EVENTS
EVENT $INITIALISATION$
then
$@act1$ $h :\in H$
end

EVENT $h1$
where
$@grd1$ $h < 23$
then
$@act1$ $h := h + 1$
end

EVENT $h2$
where
$@grd1$ $h = 23$
then
$@act1$ $h := 0$
end
end

MACHINE $M2$ REFINES $M1$
SEES C

VARIABLES h m

INVARIANTS
$@inv1$ $m \in M$
$@inv2$ $h \in H$

EVENTS
EVENT $INITIALISATION$
then
$@act1$ $h :\in H$
$@act2$ $m :\in M$
end

EVENT $h1m1$
where
$@grd1$ $h < 23$
$@grd2$ $m < 59$
then
$@act2$ $m := m + 1$
end

EVENT $h1m2$ REFINES $h1$
where
$@grd1$ $h < 23$
$@grd2$ $m = 59$
then
$@act1$ $h := h + 1$
$@act2$ $m := 0$
end

EVENT $h2m1$ REFINES $h2$
where
$@grd1$ $h = 23$
$@grd2$ $m = 59$
then
$@act1$ $h := 0$
$@act2$ $m := 0$
end

EVENT $h2m2$
where
$@grd1$ $h = 23$
$@grd2$ $m < 59$
then
$@act1$ $m := m + 1$
end
end

Figure 1.4. *Refining time from hours to minutes*

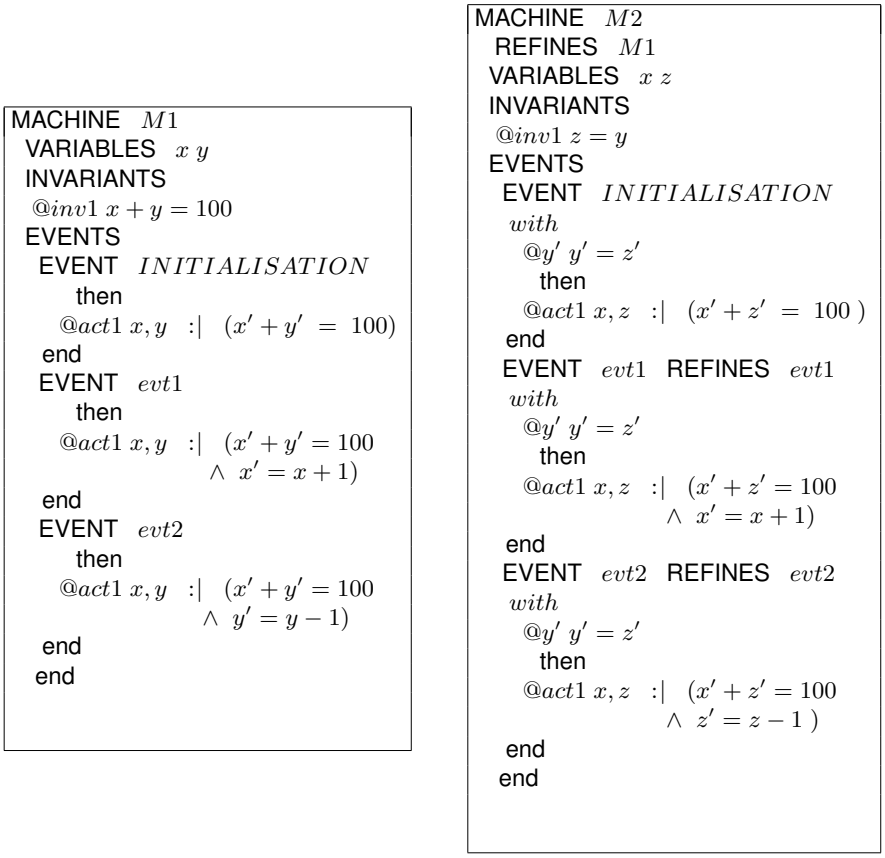


Figure 1.5. *Abstract and concrete variables*

of a concrete variable. In the figure 1.4, *h* is an abstract variable of M1 and a concrete variable in M2.

Example 12 *Abstract and concrete variables Figure 1.5 gives an example of an abstract variable y which disappears in the concrete machine in the form z. Note the role of the witness y' which links y' and z' in the initialisation of the machine M2.*

1.4.3.1. *Proof obligations for INITIALISATION*

Initialization conditions accompany and complete the diagram of Fig. 1.3 and the following verification conditions are derived: INV, SIM, FIS and WD, and, for the witness, WFIS and WWD. We make assumptions about the definition of $CInit(s, c, y')$

which can have various decomposed forms. $J(s, c, x, y)$ is a conjunction of properties $jnv_i : J_i(s, c, x, y)$ and a witness $x' : WV(s, c, y', x')$ is associated with this event (the initialisation has no parameter, so the witness does not depend on v). This simplifies the expression of the condition.

Proof Obligation INITIALISATION/jnv_i/INV

$$AX(s, c), CInit(s, c, y'), WV(s, c, y', x') \vdash J_i(s, c, x', y')$$

Proof Obligation INITIALISATION/act/SIM

$$AX(s, c), CInit(s, c, y'), WV(s, c, y', x') \vdash AInit(s, c, x')$$

A third condition is the same as for the basic case and consists in showing that the initial state exists.

Proof Obligation INITIALISATION/act/FIS

$$AX(s, c) \vdash \exists y'.CInit(s, c, y')$$

Finally, the initialisation must be well defined.

Proof Obligation INITIALISATION/act$_i$/WD

$$AX(s, c) \vdash WD(CInit_i(s, c, y'_i))$$

When the witness is not deterministic, its feasibility must be proved; in all cases, it must be well defined.

Proof Obligation INITIALISATION/x'/WFIS

$$AX(s, c), CInit(s, c, y') \vdash \exists x'.WV(s, c, y', x')$$

Proof Obligation INITIALISATION/x'/WWD

$$AX(s, c), CInit(s, c, y') \vdash WD(WV(s, c, y', x'))$$

Here again we can see the role of the witness, which really helps the work of the proof assistant and which is stated while modelling the system.

1.4.3.2. *Proof obligations for refinement: GRD, INV, SIM and WFIS*

To keep the rules as simple as possible, we assume that there are two witnesses:

– a witness for u denoted $WP(u, s, c, v, y)$

– a witness for x' denoted $WV(v, s, c, y', x')$

Existential quantifications are replaced by witnesses and allow us to derive verification conditions for the refinement with a naming specific to Rodin.

The abstract guard $G(u, s, c, x)$ is labelled $grd : G(u, s, c, x)$ and note that the concrete guard $H(v, s, c, y)$ under the conditions of the invariants $I(s, c, x)$ and $J(s, c, x, y)$ implies the abstract guard.

Proof Obligation c/grd/GRD

$$AX(s, c), I(s, c, x), J(s, c, x, y), H(v, s, c, y), WP(u, s, c, v, y) \vdash G(u, s, c, x)$$

The second verification condition also uses the second witness. Our starting point is the following property:

$$\begin{pmatrix} AX(s, c), I(s, c, x), J(s, c, x, y), \\ H(v, s, c, y), CBAP(v, s, c, y, y') \end{pmatrix} \vdash \exists x'.(\exists u.ABAP(u, s, c, x, x')) \wedge J(s, c, x', y')$$

which expresses both the preservation of the invariant J and the simulation of the abstract action by the concrete action. If we use the witnesses $WP(u, s, c, v, y)$ and $WV(v, s, c, y', x')$, we obtain the following simpler expression without existential quantification:

$$\begin{pmatrix} AX(s, c), I(s, c, x), J(s, c, x, y), \\ H(v, s, c, y), CBAP(v, s, c, y, y'), \\ WP(u, s, c, v, y), WV(v, s, c, y', x') \end{pmatrix} \vdash ABAP(u, s, c, x, x') \wedge J(s, c, x', y')$$

Then we use the conjunction property by stating two conditions, one for preserving the invariant and the other for simulating it. We recall that the gluing invariant J is the conjunction of the labelled predicates $jnv_i : J_i(s, c, x, y)$ and that the action of the abstract event a is labelled act ($act : x : |ABAP(u, s, c, x, x')$).

Proof Obligation c/jnv$_i$/INV

$$AX(s,c),\\ I(s,c,x), J(s,c,x,y),\\ H(v,s,c,y), CBAP(v,s,c,y,y'),\\ WP(u,s,c,v,y), WV(v,s,c,y',x') \\ \vdash J_i(s,c,x',y')$$

Proof Obligation c/act/SIM

$$AX(s,c),\\ I(s,c,x), J(s,c,x,y),\\ H(v,s,c,y), CBAP(v,s,c,y,y'),\\ WP(u,s,c,v,y), WV(v,s,c,y',x') \\ \vdash ABAP(u,s,c,x,x')$$

The simplification provided by the witnesses must be guaranteed and it must be shown that the conditions defined by the axioms and the two invariants guarantee the existence of the two witnesses. To do this, it is sufficient to prove that witness u and witness x' exist.

Proof Obligation c/u/WFIS

$$AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,y) \vdash \exists u.WP(u,s,c,v,y)$$

Proof Obligation c/x'/WFIS

$$AX(s,c), I(s,c,x), J(s,c,x,y),\\ H(v,s,c,y), CBAP(v,s,c,y,y') \\ \vdash \exists x'.WV(v,s,c,y',x')$$

A witness is a predicate written by the modeller, and it must be well defined like any other formal text. Rodin generates, for every witness, a well-definedness obligation whose hypotheses are those of the corresponding WFIS obligation:

Proof Obligation c/u/WWD

$$AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,y) \vdash WD(WP(u,s,c,v,y))$$

Proof Obligation c/x'/WWD

$$AX(s,c), I(s,c,x), J(s,c,x,y), \\ H(v,s,c,y), CBAP(v,s,c,y,y') \\ \vdash WD(WV(v,s,c,y',x'))$$

Advice 2 *(witnesses in Rodin) Rodin checks statically that a witness is given for every abstract parameter that disappears and for every disappearing abstract variable that the abstract event modifies non-deterministically; a missing witness is reported as an error, before any proof. The obligations* WFIS *are generated only for non-deterministic witnesses: for a deterministic witness* $u = E$ *or* $x' = E$*, Rodin substitutes* E *for the witnessed variable in the other obligations (one-point rule), so that only* WWD *remains. The obligations* WWD *are generated for all witnesses. The tool does not help in finding a witness: a wrong choice shows up only as an unprovable* GRD*,* SIM*,* INV *or* WFIS *obligation, and the modeller must then revise the witness or the gluing invariant.*

We have detailed the verification conditions that are produced by the Rodin platform and that must be proven by the proof tool. In fact, it is important to have in mind how the proof obligations are generated and what they do mean.

The concrete event must also be feasible and well defined. The obligations are those of an abstract machine, with the abstract and gluing invariants as hypotheses:

Proof Obligation c/act$_i$/FIS

$$AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,y) \vdash \exists y_i'.CBAP_i(v,s,c,y,y_i')$$

Proof Obligation c/grd$_i$/WD

$$AX(s,c), I(s,c,x), J(s,c,x,y), H_1(v,s,c,y), \ldots, H_{i-1}(v,s,c,y) \vdash WD(H_i(v,s,c,y))$$

Proof Obligation c/act$_i$/WD

$$AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,y) \vdash WD(CBAP_i(v,s,c,y,y_i'))$$

1.4.3.3. *Additional proof obligations*

The verification conditions are completed by the conditions associated with the events *convergent* or *anticipated*. Simply add the invariant $J(s, c, x, y)$ and we assume that $H(v, s, c, y)$ is the guard of the concrete event c.

Proof Obligation c/NAT

$$AX(s, c), I(s, c, x), J(s, c, x, y), H(v, s, c, y) \vdash varexp(s, c, y) \in \mathbb{N}$$

We first state the case for a convergent event c:

Proof Obligation c/VAR

$$AX(s, c), I(s, c, x), J(s, c, x, y), \\ H(v, s, c, y), CBAP(v, s, c, y, y') \\ \vdash varexp(s, c, y') < varexp(s, c, y)$$

Second, we state the case for an anticipated event c:

Proof Obligation c/VAR

$$AX(s, c), I(s, c, x), J(s, c, x, y), \\ H(v, s, c, y), CBAP(v, s, c, y, y') \\ \vdash varexp(s, c, y') \leq varexp(s, c, y)$$

Rodin generates one of these two obligations c/VAR, depending on the status of the event. For a set-valued variant, $<$ and $\leq$ become $\subset$ and $\subseteq$, and an obligation c/FIN, $AX(s, c), I(s, c, x), J(s, c, x, y), H(v, s, c, y) \vdash \text{finite}(varexp(s, c, y))$, replaces c/-NAT.

Finally, there is one last condition associated with the proof of theorems. th is the label of the theorem $SAFE(s, c, x, y)$ (see in Section 1.4.2.1).

Proof Obligation th/THM

$$AX(s, c), I(s, c, x), J(s, c, x, y) \vdash SAFE(s, c, x, y)$$

The verification conditions are expressed as sequents and are subjected in the Rodin platform to the various proof tools available, sometimes with decisive human

interaction. These additional verification conditions play a role in establishing the observation of a convergent (or helpful) event. These events are reminiscent of the helpful functions in Pnueli (**?**) or the critical actions of Méry (**?**). Their role is clearer when we consider Pnueli's induction rules or Lamport's rules for TLA, which are even more complete with fairness hypotheses. We can also mention the rule of progression by observation of a given event in the case of UNITY (**?**). For anticipated events, the idea is to develop progressively but to be able to add events across the board that won't call everything into question. In fact, when a machine is defined either by refinement or by creation, the model frame fixes the variables which cannot be modified subsequently, since the events which will be added in the next refinement must not invalidate the invariant of the refined machine, and in the case of new events, they must refine skip, i.e. not modify the variables of the refined machine. To a certain extent, it is important that all the events are introduced at the current level, but we don't know them all and the idea is to use an anticipated event which is still imprecise but which maintains the current invariant. This is the case when we are going to develop a sequential algorithm from a pre/post specification and it is quite simple to anticipate a calculation loop before delivering the result satisfying the postcondition. In this way, the anticipated event makes it possible to express that something is happening before the final computation event is observed.

1.4.3.4. *Fusion of events*

In the refinement of a machine AM by CM, the definition 16 (page 40) is reduced to this expression $\forall \mathsf{c}.\mathsf{c} \in \mathsf{E}(\mathsf{CM}) \Rightarrow \exists \mathsf{a}.\mathsf{a} \in \mathsf{E}(\mathsf{AM}) \wedge \mathsf{c}$ refines a. Every concrete event c in CM corresponds to an abstract event a and is linked by the relation refines and in a way, we could understand that the number of abstract events is less than the number of concrete events. This is not the case and it is possible to reduce the number of events per refinement by using event merging with very strong assumptions. Thus, by merging two abstract events a1 and a2, we obtain a concrete event $\mathsf{c}(\mathsf{a1} + \mathsf{a2})$ which refines each of the two events, but the condition is that the actions of the two abstract events be syntactically identical and that the two events have the same parameters. The condition is strong but allows us to merge and simplify the concrete models. Merging two events is an important mechanism for completing the Event-B refinement. This possibility requires strong conditions on the actions, which must be identical for all three events. As shown in the diagram in figure 1.6, the guards $G1$ and $G2$ are reinforced by the guard H and the verification condition is very simply the expression of this reinforcement.

Proof Obligation c/MRG

$$AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,y) \vdash G_1(u,s,c,x) \vee G_2(u,s,c,x)$$

The conditions for applying this merging are very restrictive, but we will give an example of its application. In the development of sequential algorithms (see below), we used this merge to reduce a set of events to a single event. As a result, it can be seen that during refinement either the number of events is increased or reduced, depending on the case.

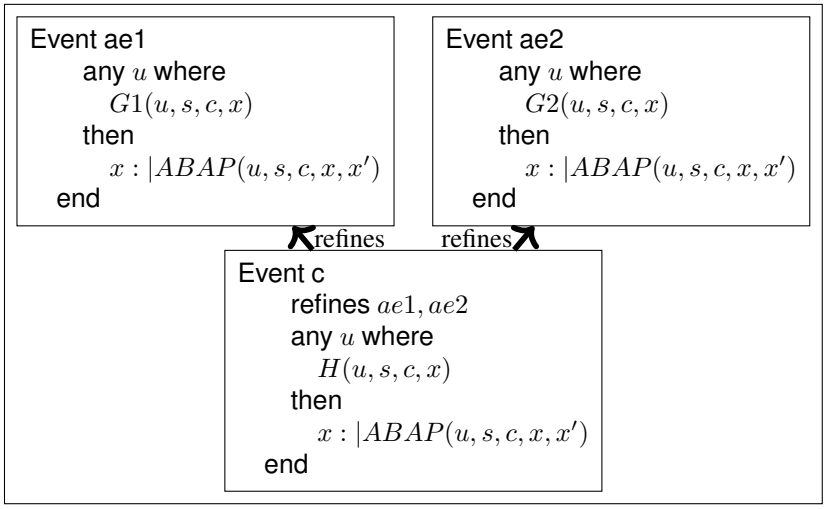


Figure 1.6. *Fusion of two events*

CONTEXT $MRG0$

CONSTANTS $a\ b$

AXIOMS
@$axm1$ $a \in \mathbb{Z} \ \wedge \ b \in \mathbb{Z}$
END

We illustrate this rule by calculating the maximum of two numbers. The context MRG0 defines the integer values a and b, and we will ensure that sup contains the larger of the two values. This machine compares two values and assigns the value of the maximum to sup. The variable $done$ controls this machine. When $done$ is TRUE, sup contains the maximum.

MACHINE $MRG1$ SEES $MRG0$
VARIABLES $i\ j\ done\ sup$
INVARIANTS
$@inv1\ i \in \mathbb{Z} \ \wedge\ j \in \mathbb{Z} \ \wedge\ done \in BOOL \ \wedge\ sup \in \mathbb{Z}$
$@inv2\ i = a \ \wedge\ j = b$
$@inv3\ done = TRUE \ \Rightarrow\ sup \geq i \ \wedge\ sup \geq j \ \wedge\ sup \in \{a, b\}$
EVENTS
EVENT $INITIALISATION$
then
$@act1\ i := a$
$@act2\ j := b$
$@act3\ done := FALSE$
$@act4\ sup :\in \mathbb{Z}$
end
EVENT $e1$
where
$@grd1\ i < j$
$@grd2\ done = FALSE$
then
$@act1\ sup := j$
$@act2\ done := TRUE$
end
EVENT $e2$
where
$@grd1\ i \geq j$
$@grd2\ done = FALSE$
then
$@act1\ sup := i$
$@act2\ done := TRUE$
end
end

Before applying the MRG rule, we must construct a refinement of this machine that satisfies the application conditions. This means refining e1 and e2 so that the action is identical for each event. The machine MRG2 (see Fig. 1.7) refines the machine MRG1, and the actions of the two events are identical. We can apply the rule for merging the two events and construct an event merge(e1,e2) which corresponds to a conditional instruction. Figure 1.8 illustrates the application of this rule and gives a view of the events. The proof is fairly easy, since the obligation c/MRG, $done = FALSE \vdash (i < j \wedge done = FALSE) \vee (i \geq j \wedge done = FALSE)$, reduces to $i < j \vee i \geq j$, which is true.

MACHINE $MRG2$
REFINES $MRG1$
SEES $MRG0$
VARIABLES $i\ j\ done\ sup$
EVENTS
EVENT $INITIALISATION$
then
$@act1\ i := a$
$@act2\ j := b$
$@act3\ done := FALSE$
$@act4\ sup :\in \mathbb{Z}$
end
EVENT $e1$ REFINES $e1$
where
$@grd1\ i < j$
$@grd2\ done = FALSE$
then
$@act1\ sup, done :| (done = FALSE \wedge (i < j \Rightarrow sup' = j) \wedge (i \geq j \Rightarrow sup' = i) \wedge done' = TRUE))$
end
EVENT $e2$ REFINES $e2$
where
$@grd1\ i \geq j$
$@grd2\ done = FALSE$
then
$@act1\ sup, done :| (done = FALSE \wedge (i < j \Rightarrow sup' = j) \wedge (i \geq j \Rightarrow sup' = i) \wedge done' = TRUE)$
end
end

MACHINE $MRG3$
REFINES $MRG2$
SEES $MRG0$
VARIABLES $i\ j\ done\ sup$
INVARIANTS
$theorem\ @inv1\ done = TRUE \Rightarrow sup \geq i \wedge sup \geq j \wedge sup \in \{a, b\}$
$theorem\ @inv2\ i = a \wedge j = b$

EVENTS
EVENT $INITIALISATION$
then
$@act1\ i := a$
$@act2\ j := b$
$@act3\ done := FALSE$
$@act4\ sup :\in \mathbb{Z}$
end

EVENT $merge(e1, e2)$
REFINES $e1\ e2$
where
$@grd2\ done = FALSE$
then
$@act1\ sup, done :| (done = FALSE \wedge (i < j \Rightarrow sup' = j) \wedge (i \geq j \Rightarrow sup' = i) \wedge done' = TRUE)$
end
end

Figure 1.7. *Merging events*

This merging rule will be used in the chapter entitled *Design of Correct by Construction Sequential Algorithms*, when developing sequential algorithms. Refinement is an operation that allows new events to be introduced as well as reinforcing existing events, and in the case of this rule, it allows the number of events to be reduced.

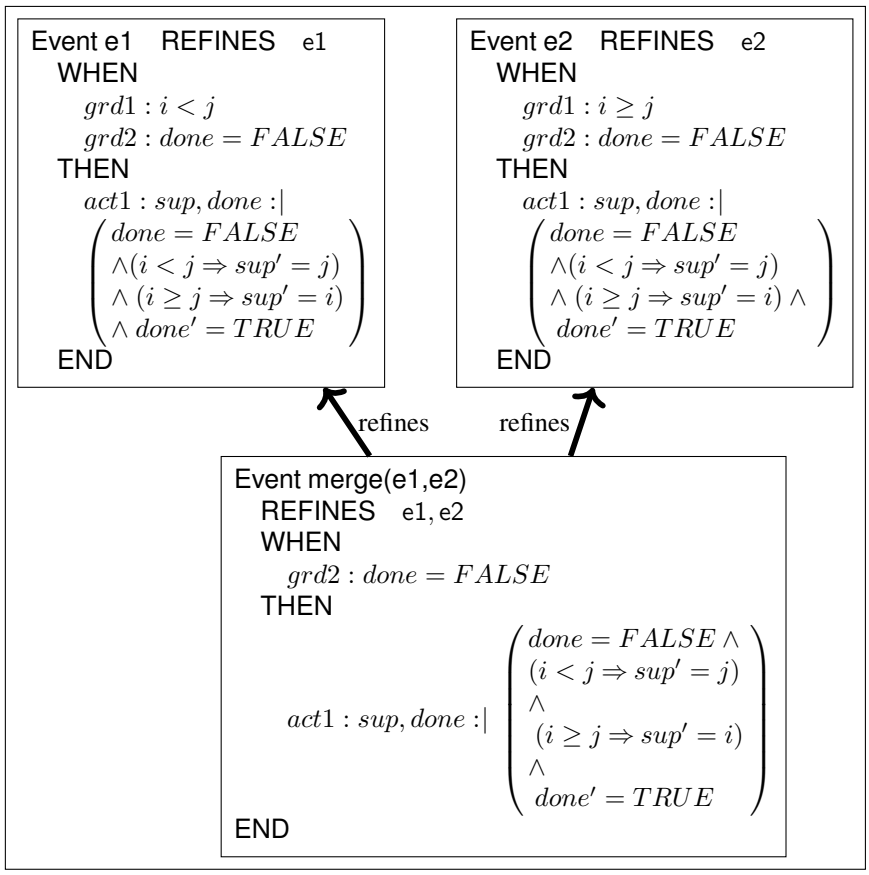


Figure 1.8. *Fusion of two events e1 and e2*

1.4.3.5. *Semantics of Event-B*

The semantics of Event-B has been addressed in numerous studies and, to a certain extent, proof obligations constitute its logical expression. Invariance and safety properties are central, and the semantics ensure that these properties are preserved throughout the refinement process. However, there remains an important property to consider when defining an Event-B model (machine or refinement) namely deadlock

freedom. In our presentation, we have neglected that property both for an initial abstract machine AM and for a refinement machine CM. Thus, for an initial abstract machine, the logical context must be consistent and the initial conditions of this machine must be satisfiable. It is inconceivable to define initial conditions corresponding to the absence of initial values. A second element to be verified corresponds to the concept of deadlock freedom.

The first proof obligation, called m/DLF, states that at least one guard is true under the assumption of the invariant $I(s, c, x)$.

Proof Obligation m/DLF

$$AX(s, c), I(s, c, x) \vdash \left(\bigvee_{\mathsf{e} \in \mathsf{E(AM)}} \exists u_{\mathsf{e}}.G_{\mathsf{e}}(u_{\mathsf{e}}, s, c, x) \right)$$

The second proof obligation, called r/DLF states that the disjunction of abstract guards implies the disjunction of concrete guards, or equivalently that the concrete events of the machine CMdo not deadlock more often than the abstract events of AM.

Proof Obligation r/DLF

$$AX(s, c), I(s, c, x), J(s, c, x, y) \left(\bigvee_{\mathsf{e} \in \mathsf{E(AM)}} \exists u_{\mathsf{e}}.G_{\mathsf{e}}(u_{\mathsf{e}}, s, c, x) \right) \vdash \left(\bigvee_{\mathsf{e} \in \mathsf{E(CM)}} \exists v_{\mathsf{e}}.H_{\mathsf{e}}(v_{\mathsf{e}}, s, c, y) \right)$$

These two proof obligations complement the *logical* semantics of Event-B as expressed in the form of the proof obligations listed in this document. However, DLF proof obligations are not systematically generated as theorems using the tools Rodin and Atelier B and require a sufficiently expressive invariant to complete the proofs. This means that whether the deadlock-freedom property is taken into account depends on the system being modelled, and that system may deadlock depending on requirements. The deadlock-freedom property therefore becomes optional and is generally verified using a model-checking tool such as ProB. To conclude the discussion on the deadlock-freedom property, the book (**?**, chapter 16, p. 489, subsection 4.2) shows that this property is essential for ensuring the safety property SAF-1, which states that *No persons must remain trapped in a location*. Several solutions are considered, in order to guarantee it, and a solution is required to ensure that no one remains trapped in a building. The two solutions of J.-R. Abrial are either to require that any person authorized to be in a location should be authorized to be in a location communicating with the first one (SAF-2), or that any person authorized to be in a location should be authorized to be in a location communicating with the *outside* location (SAF-3). The

deadlock-freedom property helps to develop a safety analysis of general systems, and both solutions correspond to practical solutions implemented in operational control systems.

As mentioned in Section 1.2, an Event-B model observes the state of a system through flexible variables, and the semantics of an Event-B model can be defined by state traces generated by the transition relations of that model. Schneider et al. (**?**) have established a behavioural semantics for Event-B based on CSP. They express Event-B refinement using the CSP refinement.

Chapter 14 of (**?**) presents the mathematical models of the proof obligations described in the previous chapters of the same work (**?**). Proof obligations are described in terms of backward and forward simulations, whilst we have retained the expression given in the context of the wp calculus (see property 4 and definition 15). Definition 15 and Property 9 focus on the use of a definition of refinement which J.-R. Abrial had called condition 4 in (**?**, p. 526) and which made it possible to guarantee condition 3 on the refinement relation defined by the relation between the wp of the two refined actions ($a \sqsubseteq c$ means that $\forall P.[a]P \Rightarrow [c]P$). In our presentation, we have sought to highlight the link between the two styles and demonstrate the importance of wp calculus. The explanations provided are proofs demonstrating the equivalence of the two approaches, and reference may be made to the summary table given (see (**?**, p. 530)) in the form of four obligations. Both approaches have been used in the Atelier B and Rodin tools.

In the book *Abrial Gedenkschrift*, P. Cousot (**?**) reformulates the ideas developed in Event-B and provides a very precise description of the temporal modality associated with the concept of a variant. In this document, P. Cousot analyses the relationships between abstract models and concrete models with very precise references to (**?**) and arrives at an expression of the duality between refinement and abstract interpretation. Finally, the text arrives at a general principle of induction from a trace semantics.

1.5. Overview of the Event-B Modelling Language

We take up the elements of the Event-B language by developing a simplified abacus system, in order to illustrate the modelling language, refinement, ordinary, anticipated and convergent events, witnesses, invariants and safety properties. Other examples will be developed in the following chapters and will illustrate the modelling possibilities of Event-B .

1.5.1. *Playing with Event-B*

We are dealing with a simple example to illustrate the various elements we have already presented, but it is clear that the other chapters will be illustrations of the use

of the Event-B language with the Rodin platform or the Atelier B platform. In fact, we have an idea of a simple system for calculating the *addition* function, and this method is the one that the school teacher taught, when the author was 6 years old and discovering numbers and calculations. Figure 1.9 shows contexts and machines used to describe the rules for using the *abacus*. The abacus can be used either by moving balls from top to bottom in one move (rule 1), or by moving them one after the other (rule 2). The abacus uses potential energy, as the balls will descend very simply once they have been released. Initialization is done quite simply by inverting the top and bottom. Formally, the problem to solve can be defined as follows using a contract-based notation.

variables x, y, r, ok

requires
$x_0 \in \mathbb{N}$
$y_0 \in \mathbb{N}$
$ok_0 \in BOOL$

ensures
$r_f = x_0 + y_0$
$ok_f = TRUE$

The contract expresses a relationship between the initial values of x and y, denoted x_0 and y_0, and the final value of r denoted r_f. This relationship simply expresses that the final value of the variable r denoted r_f is the sum $x_0 + y_0$. Obviously, we use the mathematical operator $+$ and we aim to construct, by refinement, a set of events modelling the movements of the abacus and calculating the operator $+$. Two contexts are mentioned in the figure 1.9 namely C0 and C1. C0 is the context for defining the two natural numbers a and b used for the computation; they correspond to the initial values x_0 and y_0 of the contract and are the *inputs* for short.

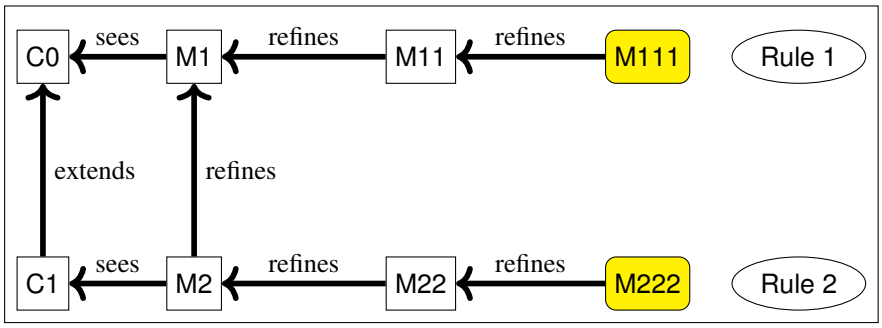


Figure 1.9. *Organisation of refinements for ABACUS*

a and b are two natural numbers and are defined in context C0. In context C1, we define balls of the abacus and the two sets of balls representing sets with cardinalities corresponding to a and b.

CONTEXT $C0$

CONSTANTS $a\ b$

AXIOMS
$@axm1: a \in \mathbb{N}$
$@axm2: b \in \mathbb{N}$
END

CONTEXT $C1$ EXTENDS $C0$

$sets\ B$

CONSTANTS $seta\ setb\ ball$

AXIOMS
$@axm1: B \neq \varnothing \land finite(B)$
$@axm5: seta \subseteq B \land card(seta) = a$
$@axm2: setb \subseteq B \land card(setb) = b$
$@axm3: seta \cap setb = \varnothing$
$thm\ @axm4: finite(seta) \land finite(setb)$
$@axm6: ball \in B$
$@axm7: \forall u, v \cdot$
$u \subseteq B \land v \in B \land v \notin u$
$\Rightarrow$
$card(u \setminus \{v\}) = card(u) - 1$
END

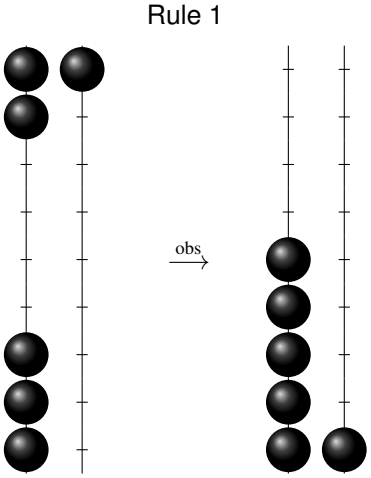
The aim of this modelling is to show the link between the mathematical function + and its calculation using two calculation rules, using variable refinement and abstraction. The representation of a number n on the abacus is a set of n balls. Figure 1.9 gives contexts and machines modelling the two rules, and we are detailing the different components. Our abacus has two rows of balls. The first row is used to calculate addition by moving the balls from top to bottom. The second row is a control structure and has a single ball that is at the top at the start of the calculation and at the bottom when the calculation is complete. The vertical positioning of the rows eliminates the need for an energy source, and the calculation rules are straightforward: the balls at the top of the first row move to the bottom of the first row. Figure 1.10 summarizes the Event-B components that we have produced for modelling the two rules according to the figure 1.9.

1.5.1.1. *Rule 1 of the abacus*

Rule 1 allows the balls to be moved from the top to the bottom in a single move. This means that a $2 + 3$ addition is modelled as follows: three balls are placed at the bottom and two at the top of the first row, and one ball is placed at the top of the second row. The calculation is observed when the two balls are released, together with the control ball of the second row.

Components	Explanations of components
C0 , C1	The two contexts define the two natural numbers a and b to use for the addition. The set of balls B is defined and two sets seta and setb are defined for representing respectively a and b: a (respectively b) is denoting the cardinality of seta (respectively setb).
M1	The event calling_a_function is modelling the postcondition and uses two variables r and ok where r gets the result of the addition and ok gets the value TRUE to mean that the variable r is updated. The event computing is anticipating the detailed intermediate computations which are detailed in the refinement M11.
M11	Two new variables setr, setok are introduced and model numbers or booleans using balls. When cardinality of setr is n, it means that setr represents n. An event obs refines the event calling_a_function and simulates the computation by merging the balls into setr.
M111	The refinement M111 hides the variables r and ok and it keeps visible the variables with balls. The event obs is observing the rule 1 and computes the addition as the union of balls.
M2	In the machine M2, two intermediate variables x and y containing a and b are introduced and model the state of the top and bottom of the first row. The event calling2 refines the event calling_a_function and the new event step2 is refining the event computing; it simulates an iteration. Events are simulating the addition function.
M22	The variables setx, sety, setr and setok are introduced to represent variables containing integers or booleans. Events are enriched to handle the new variables.
M222	The variables r, ok, x and y are hidden and the events are modified accordingly. The rule 2 is simulated by the event step3.

Figure 1.10. *Sketch for contexts, machines and refinements for abacus*



The event obs triggers the mechanism for passing the two top balls onto the three bottom balls. The result is obtained by counting the number of lower balls. We could use the gaseous method, which would lead us to observe that the lower balls rise. Whatever process is used, it is completed when the control ball is at the bottom (or at the top, with the gaseous method). Rule 1 for using the abacus is to move the balls from the top to the bottom and to update an indicator showing that the rule has been applied once and only this rule is applied.

The machine M1 (see Fig. 1.11), which sees the context C0, expresses the contract of the addition function as the event calling_a_function: after this event, the variable ok is set to TRUE and r is set to $a + b$. The event computing *anticipates* a hidden computation process. The refinement machine M11 (see Fig. 1.11) introduces two new variables, setr and setok, which model the rows of the abacus by sets of balls; its invariant expresses the relationship between ok, r, setr and setok. M11 is an intermediate machine, refined by M111 (see Fig. 1.12), where the variables ok and r are hidden. The event obs of M111 models the movement of the balls from top to bottom in a single move, and thus simulates rule 1; this machine also illustrates the use of witnesses in the INITIALISATION event.

Rule 1 calculates the addition in one move and the refinement expresses that the + function is calculated according to this rule application.

The computing event is undoubtedly one that may seem like too much, and it could disappear at this level of refinement. In fact, it models the fact that there may be non-visible calculations that prepare the final result. In fact, it anticipates intermediate calculations which are not visible but which could exist. In all cases, the invariants remain verified.

1.5.1.2. *Rule 2 of the abacus*

Rule 2 allows the balls to be moved from the top to the bottom, but one ball at a time. So, a $2 + 3$ addition is modelled as follows: three balls are placed at the bottom and two at the top of the first row, and one ball is placed at the top of the second row. The calculation is observed when the first ball is released, then the second ball is released and finally the single ball in the second row is released.

MACHINE $M1$ SEES $C1$

VARIABLES $r\ ok$

INVARIANTS
$@inv1\ r \in \mathbb{Z}$
$@inv2\ ok \in BOOL$
$@inv3\ ok = TRUE$
$\Rightarrow\ r = a + b$

EVENTS
EVENT $INITIALISATION$
then
$@act3\ ok := FALSE$
$@act4\ r :\in \mathbb{Z}$
end

EVENT $calling_a_function$
where
$@grd1\ ok = FALSE$
then
$@act1\ ok := TRUE$
$@act2\ r := a + b$
end

$anticipated$ EVENT $computing$
then
$@act1\ r, ok :|$
$(ok' \in BOOL \wedge r' \in \mathbb{Z}$
$\wedge$
$(ok' = TRUE \Rightarrow r' = a + b))$
end
end

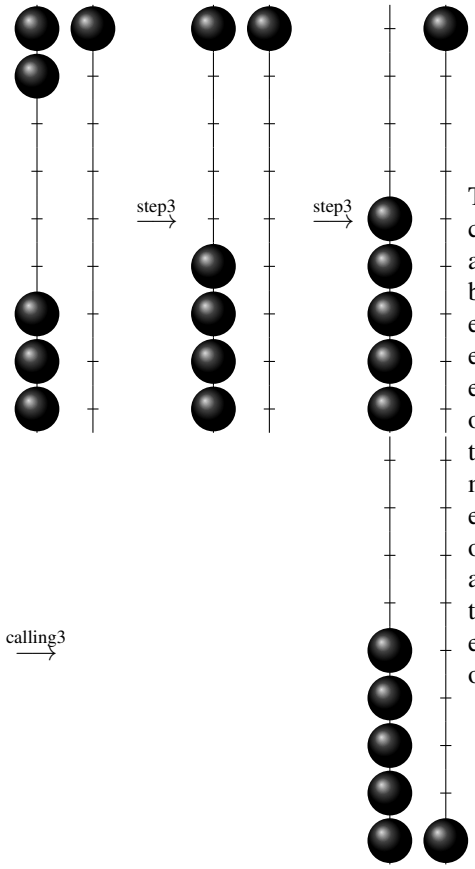
MACHINE $M11$
REFINES $M1$
SEES $C1$
VARIABLES $r\ ok\ setr\ setok$
INVARIANTS
$@inv1\ setr \subseteq B \wedge setok \subseteq B$
$@inv5\ setok = \{ ball \} \Rightarrow setr = seta \cup setb$
$@inv2\ setok = \{ ball \} \Rightarrow ok = TRUE$
$@inv3\ ok = TRUE \Rightarrow setok = \{ ball \}$
$@inv6\ card(setr) = r$
$@inv7\ setok \subseteq \{ ball \}$
EVENTS
EVENT $INITIALISATION$
then
$@act4\ r, setr, ok, setok :|$
$(r' = 0 \wedge ok' = FALSE$
$\wedge\ setok' = \varnothing \wedge setr' = \varnothing)$
end
EVENT obs REFINES $callingafunction$
where
$@grd1\ ok = FALSE$
$@grd2\ setok = \varnothing$
then
$@act1\ ok := TRUE$
$@act2\ r := a + b$
$@act3\ setr := seta \cup setb$
$@act4\ setok := \{ ball \}$
end

$anticipated$ EVENT $computing$
REFINES $computing$
then
$@act1\ r, ok, setr, setok :|$
$(ok' \in BOOL \wedge r' \in \mathbb{Z}$
$\wedge\ setok' \subseteq \{ ball \}$
$\wedge\ (setok' = \{ ball \}$
$\Rightarrow\ ok' = TRUE$
$\wedge\ setr' = seta \cup setb \wedge r' = a + b)$
$\wedge\ (ok' = TRUE$
$\Rightarrow\ r' = a + b$
$\wedge\ setok' = \{ ball \}$
$\wedge\ setr' = seta \cup setb)$
$\wedge\ card(setr') = r')$
end
end

Figure 1.11. *Case of the rule 1 of abacus (I)*

MACHINE $M111$ REFINES $M11$ SEES $C1$
VARIABLES $setr\ setok$
INVARIANTS
$theorem\ @inv1\ setok = \{ ball \} \Rightarrow setr = seta \cup setb$
EVENTS
EVENT $INITIALISATION$
$with$
$@r'\ r' = 0$
$@ok'\ ok' = FALSE$
then
$@act4\ setr, setok :| (setok' = \varnothing \wedge setr' = \varnothing)$
end
EVENT obs REFINES obs
where
$@grd2\ setok = \varnothing$
then
$@act3\ setr := seta \cup setb$
$@act4\ setok := \{ ball \}$
end
$anticipated$ EVENT $computing$ REFINES $computing$
$with$
$@r'\ r' = card(setr')$
$@ok'\ (setok' = \{ ball \} \Rightarrow ok' = TRUE) \wedge (setok' = \varnothing \Rightarrow ok' = FALSE)$
then
$@act1\ setr, setok :| (setok' \subseteq \{ ball \} \wedge (setok' = \{ ball \} \Rightarrow setr' = seta \cup setb))$
end
end

Figure 1.12. *Case of the rule 1 of abacus (II)*



The M1 machine is refined by the M2 machine and introduces two new variables, x and y, which allow an iteration controlled by the value of y to be implemented. A new event step2 defines the computation or iteration step. The invariant $x + y = a + b$ expresses the maintenance of the number of balls. The variant y is used to express the termination of the process. Then the machine M22 introduces the balls and the event step3 observes both the movement of a ball and the updating of the variables x and y. Then a final refinement M222 hides the variables r, x, y and ok, and we find the event step3, which models the movement of one ball at a time.

MACHINE $M2$
REFINES $M1$
SEES $C0$
VARIABLES $x\ y\ r\ ok$
INVARIANTS
$@inv1\ x \in \mathbb{N}$
$@inv2\ y \in \mathbb{N}$
$@inv3\ x + y = a + b$
$@inv4\ ok = TRUE$
VARIANT y
EVENTS
EVENT $INITIALISATION$
then
$@act1\ ok := FALSE$
$@act2\ r :\in \mathbb{Z}$
$@act3\ x := a$
$@act4\ y := b$
end
EVENT $calling2$
REFINES $calling_a_function$
where
$@grd1\ ok = FALSE$
$@grd2\ y = 0$
then
$@act1\ ok := TRUE$
$@act2\ r := x$
end
$convergent$ EVENT $step2$
REFINES $computing$
where
$@grd1\ ok = FALSE$
$@grd2\ y \neq 0$
then
$@act1\ x := x + 1$
$@act2\ y := y - 1$
end
end

The refinement introduces two new variables x and y which are used to introduce the iterative process starting with y containing b and ending when y contains 0. The property of this process is that there is only one event which observes the decay of y and which is a convergent event.
The ok variable is used to express that the calculation process is complete when the value of y is 0. The end of the process is detected by the value of y and convergence is expressed by the variant y which decreases strictly when the event step2 is observed. The proofs are simple and the property $x + y = a + b$ is inductive.
The refinement of M2 into M22 amounts to materialising the iterative process with balls and the M22 machine introduces four new variables setr, setok like M11 and setx and sety for the two sides of the abacus. The preservation of $x + y = a + b$ is ensured by the fact that no one can add or remove balls. One could add demonic events that lose or add balls; such events would violate the property $setx \cup sety = seta \cup setb$.
A final refinement, M222, of M22 consists in hiding the variables r,x,y,ok, thus obtaining an iterative process which can only apply rule 2.

MACHINE $M22$ REFINES $M2$ SEES $C1$
VARIABLES $r\ ok\ setx\ sety\ x\ y\ setr\ setok$
INVARIANTS
$@inv1\ setx \subseteq B$
$@inv2\ sety \subseteq B$
$@inv3\ setr \subseteq B \land setok \subseteq B \land setok \subseteq \{ ball\}$
$@inv4\ setx \cap sety = \varnothing \land setx \cup sety = seta \cup setb$
$@inv5\ setok = \{ ball\} \Rightarrow sety = \varnothing \land setr = seta \cup setb$
$@inv6\ (setok = \varnothing \Rightarrow ok = FALSE) \land (ok = FALSE \Rightarrow setok = \varnothing)$
$@inv7\ (setok = \{ ball\} \Rightarrow ok = TRUE) \land (ok = TRUE \Rightarrow setok = \{ ball\})$
$@inv8\ x = card(setx) \land y = card(sety)$
EVENTS
EVENT $INITIALISATION$
then
$@act1\ ok := FALSE$
$@act2\ r :\in \mathbb{Z}$
$@act3\ x, y, setx, sety, setr, setok :|$
$(setok' = \varnothing \land setr' \subseteq B \land setx' = seta \land sety' = setb \land x' = a \land y' = b)$
end
EVENT $calling3$ REFINES $calling2$
where
$@grd1\ ok = FALSE$
$@grd2\ setok = \varnothing$
$@grd3\ sety = \varnothing$
then
$@act1\ ok := TRUE$
$@act2\ r := x$
$@act3\ setr := setx$
$@act4\ setok := \{ ball\}$
end
EVENT $step3$ REFINES $step2$
any z
where
$@grd1\ ok = FALSE$
$@grd2\ sety \neq \varnothing$
$@grd3\ z \in sety$
$@grd4\ setok = \varnothing$
$@grd5\ y \neq 0$
then
$@act1\ setx := setx \cup \{ z\}$
$@act2\ sety := sety \setminus \{ z\}$
$@act3\ x := x + 1$
$@act4\ y := y - 1$
end
end

MACHINE $M222$ REFINES $M22$ SEES $C1$
VARIABLES $setx\ sety\ setr\ setok$
EVENTS
EVENT $INITIALISATION$
$with$
$@x'\ x' = a$
$@y'\ y' = b$
then
$@act7\ setx, sety, setr, setok\ :|$
$(setok' = \varnothing \wedge setr' \subseteq B \wedge setx' = seta \wedge sety' = setb)$
end
EVENT $calling3$ REFINES $calling3$
where
$@grd3\ setok = \varnothing$
$@grd4\ sety = \varnothing$
then
$@act3\ setr := setx$
$@act4\ setok := \{ ball \}$
end
EVENT $step3$ REFINES $step3$
any z
where
$@grd2\ sety \neq \varnothing$
$@grd3\ z \in sety$
$@grd4\ setok = \varnothing$
then
$@act1\ setx := setx \cup \{ z \}$
$@act2\ sety := sety \setminus \{ z \}$
end
end

Rule 2 allows the addition to be calculated in several moves and the refinement expresses that the + function is calculated according to this application of the rule. This is an iteration that is completed when there is a ball in the setok variable.

We have used Event-B modelling to describe the rules for using the abacus to calculate addition, and we have also verified these rules against the process of using the abacus. This example shows how Event-B can be used to describe reactive systems incrementally.

1.6. Conclusion

We have presented elements of the Event-B language, which is based on B but offers a simple way to express transitions. A machine in Event-B describes the state of an *observed* system by listing variables and asserting an invariant. It also allows for changes in variables to be expressed by a finite list of events. The machine is checked by verifying a list of verification conditions expressing the preservation of the invariant by the events. The data description uses set theory and the predicate calculus of the B language (**?**) with some differences (**?**).

Event-B was developed from the ground up based on the foundational work of Jean-Raymond Abrial's presentation (**?**) at the inaugural conference (**?**) and the contributions of Ralph Back and Reino Kurki-Suonio (**?**). The choices made have enabled us to use the Atelier B tool to develop the first Event-B models. Finally, it should be noted that the refinement of events, with new events refining skip, witnesses and event statuses, is an original element of this approach, implemented in the Rodin platform. The Event-B language is designed for developing models of reactive systems using an incremental and progressive approach guided by a set of techniques such as proof, animation and simulation supported by Rodin. However, there is still a learning phase to use this language and the following chapters will propose such techniques.